LAPORAN PRAKTIKUM

POSTTEST 8

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Nauvan Hadya Satrya (2609106014)

Kelas (A1 ‘26)

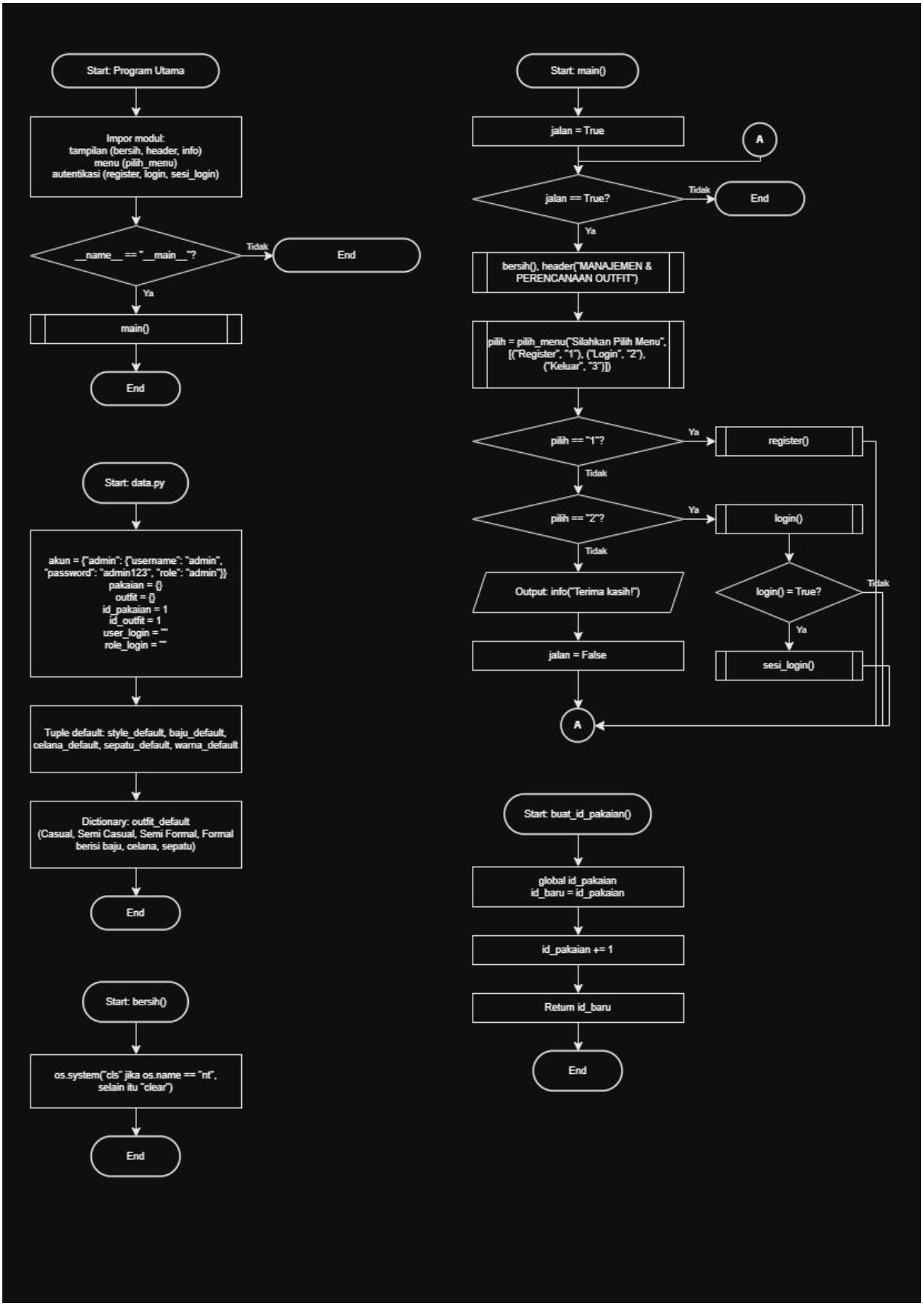
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

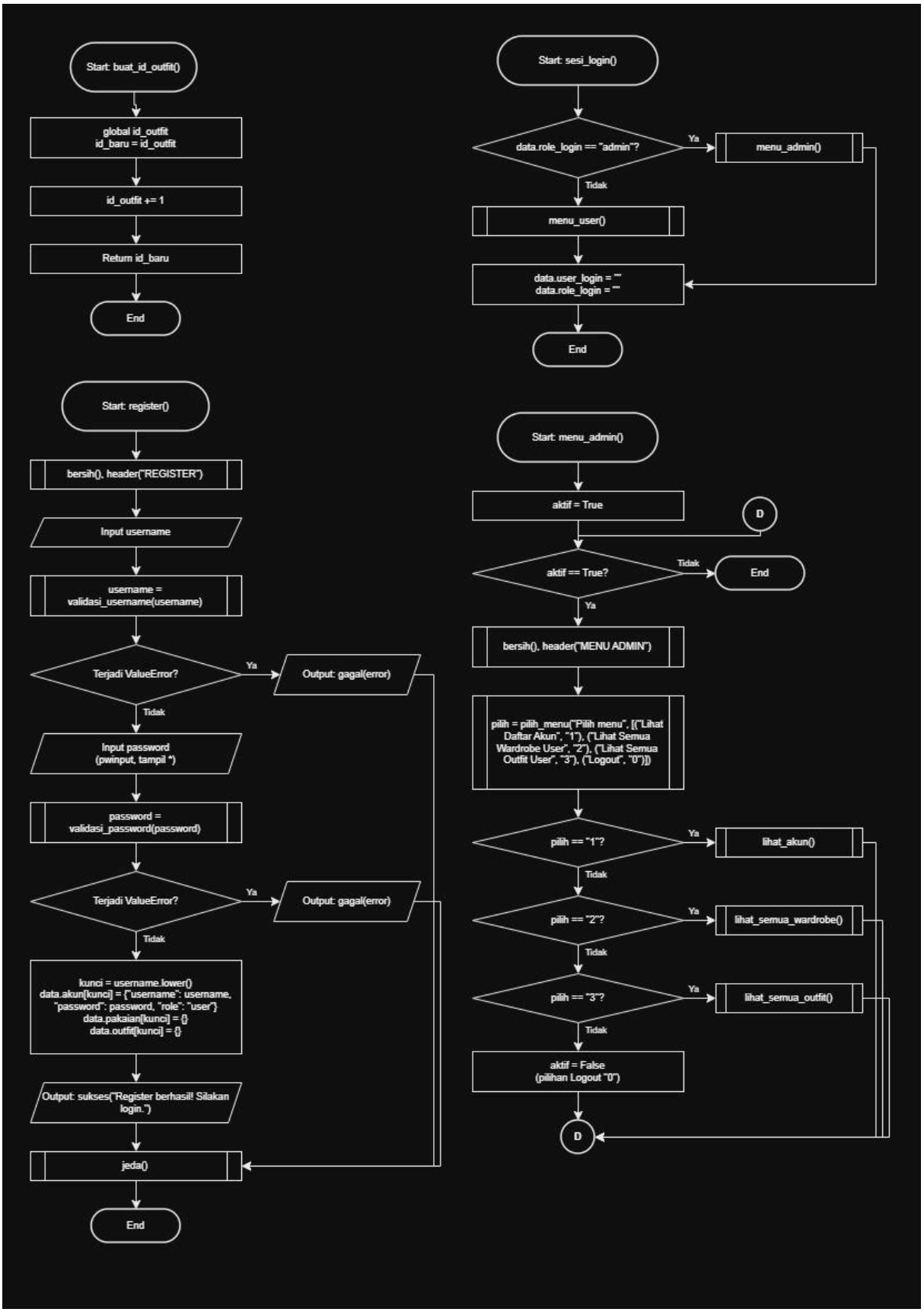
SAMARINDA

2026

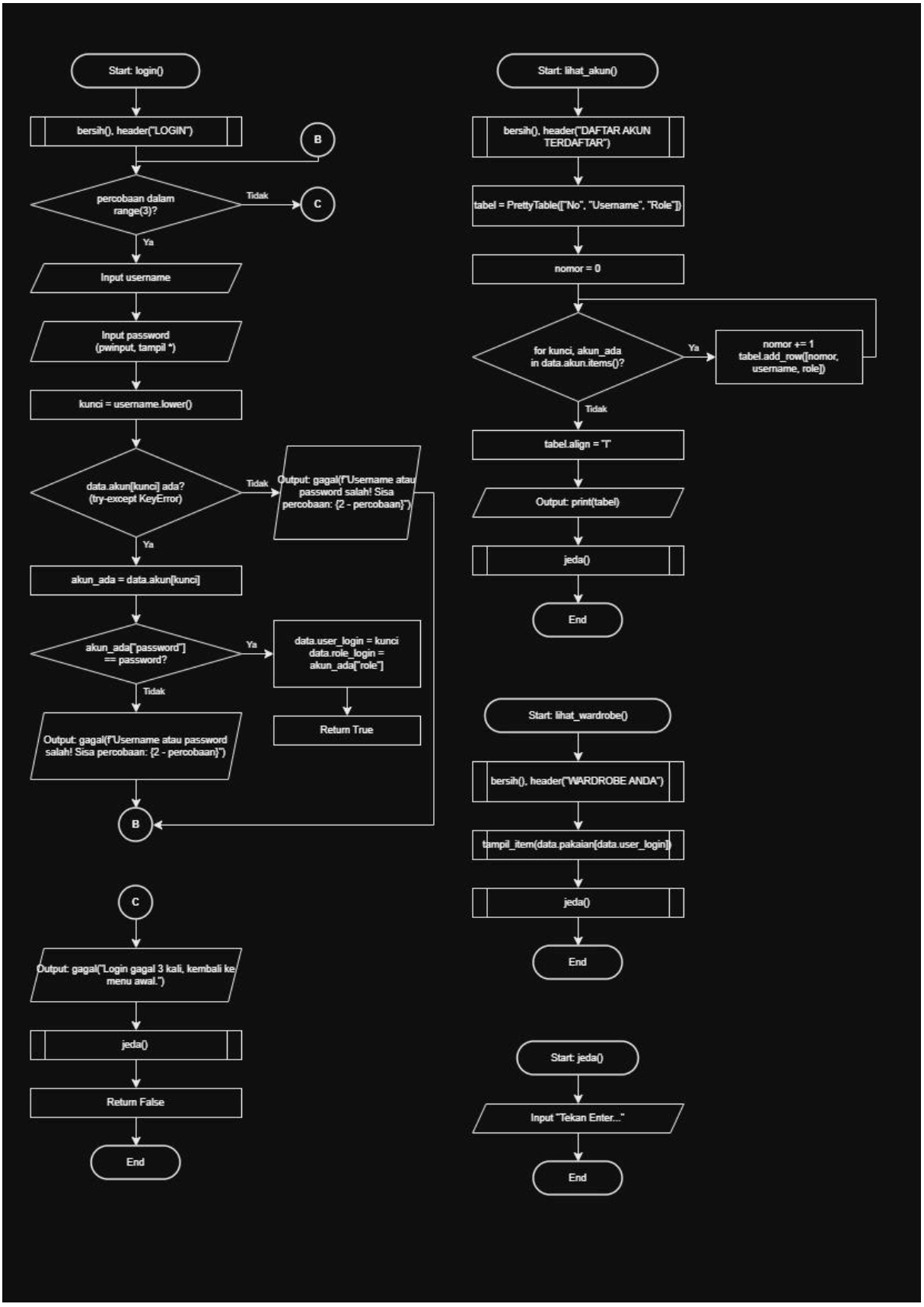
1. Flowchart



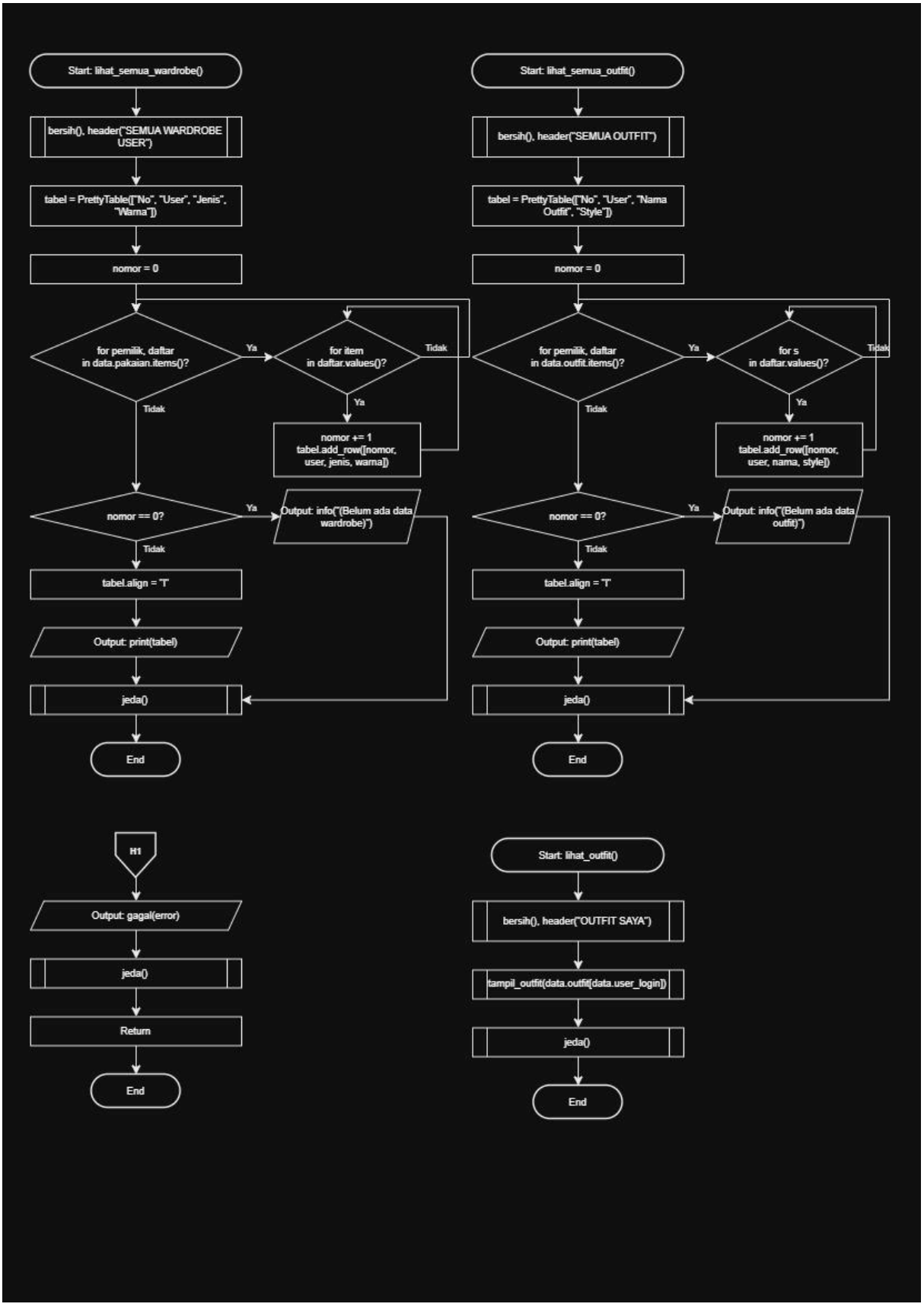
Gambar 1.1 Flowchart Halaman 1



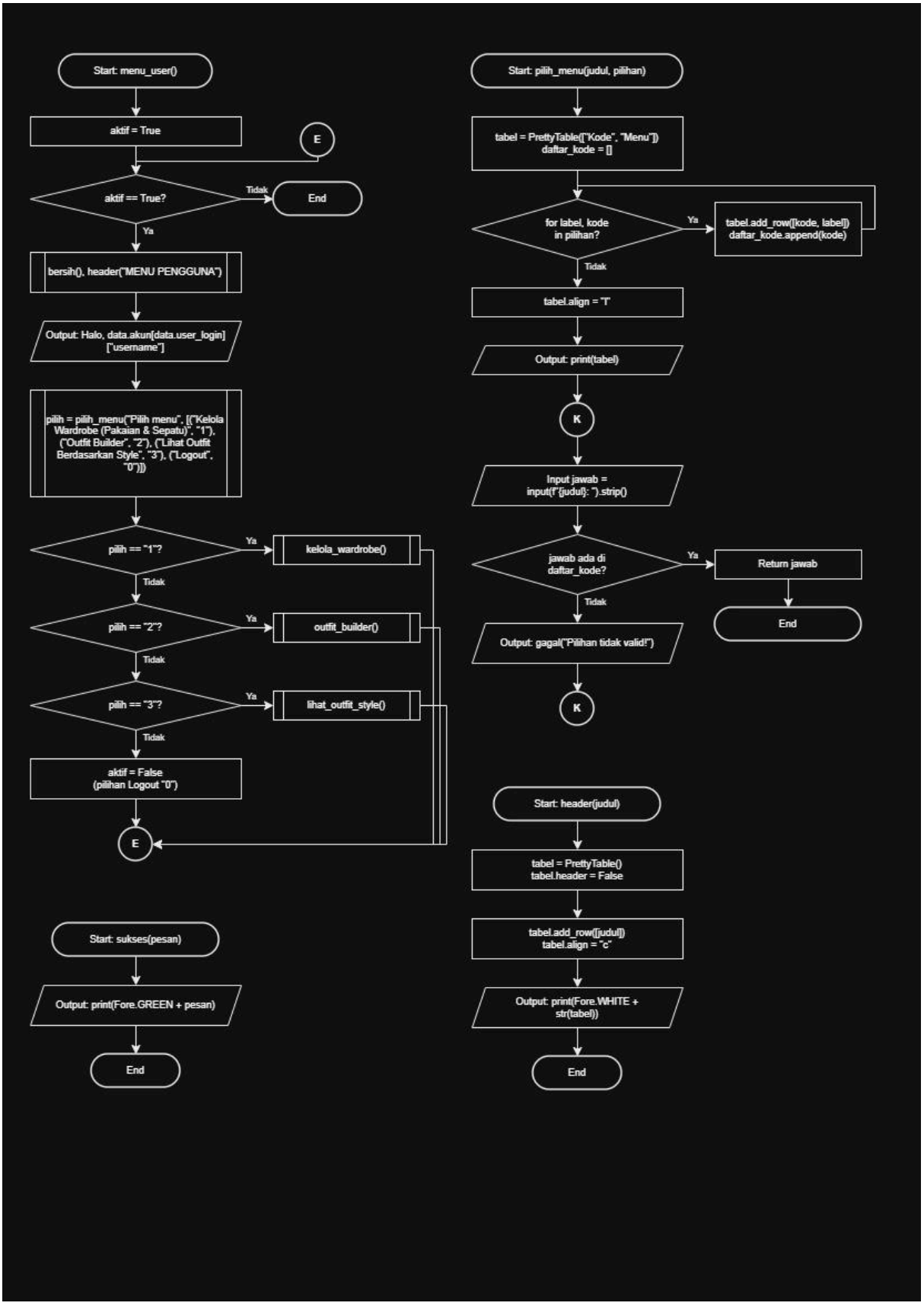
Gambar 1.2 Flowchart Halaman 2



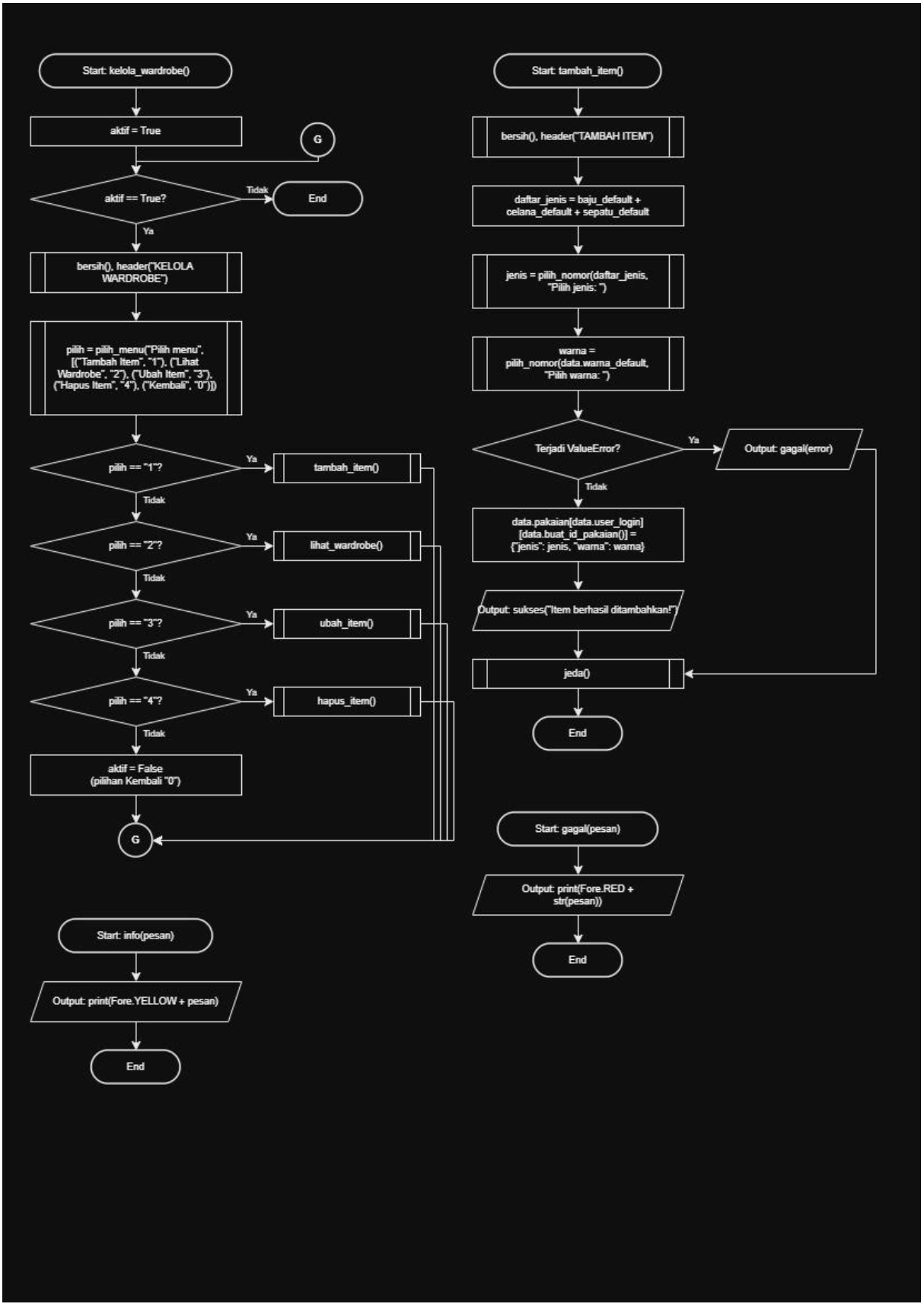
Gambar 1.3 Flowchart Halaman 3



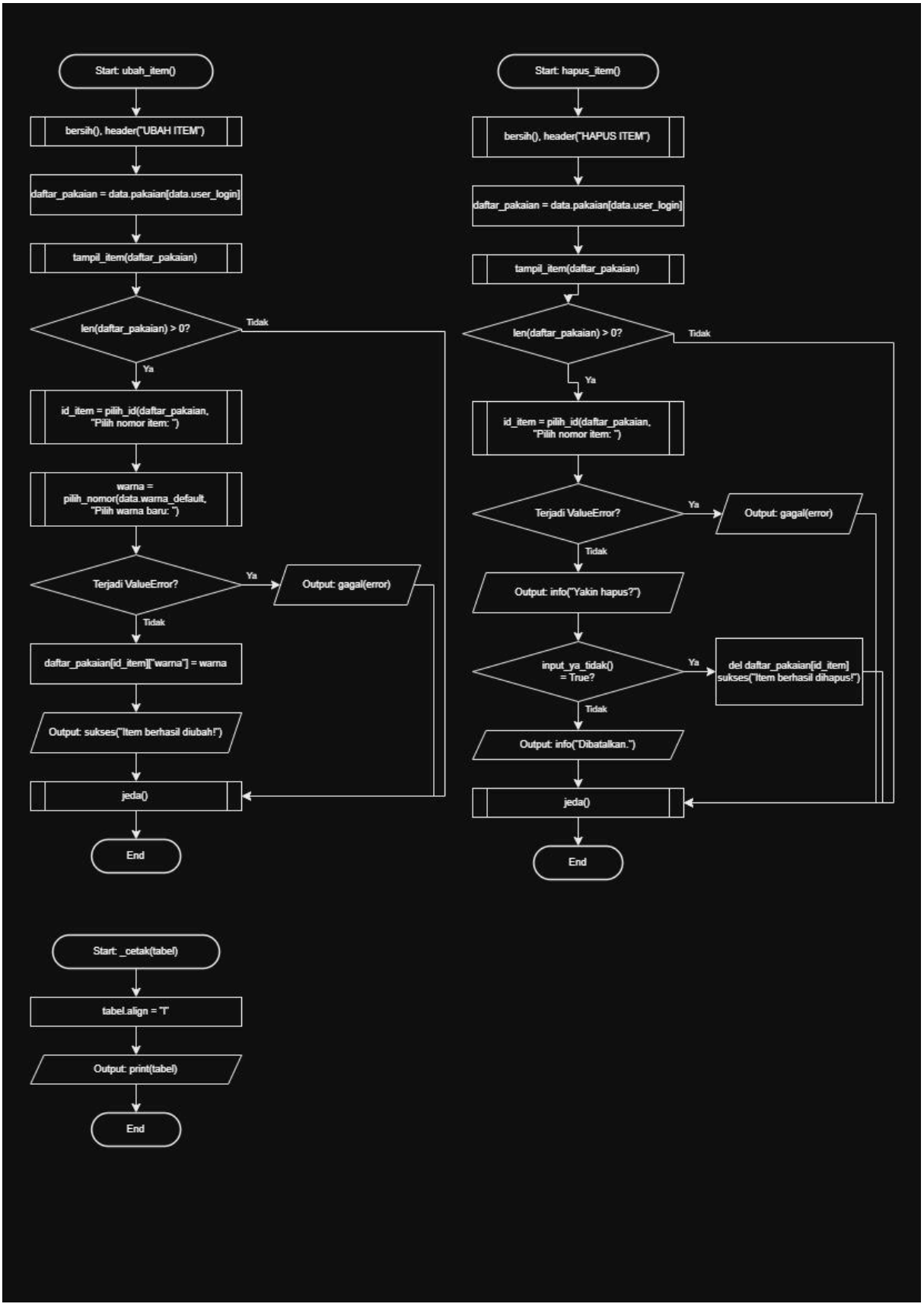
Gambar 1.4 Flowchart Halaman 4



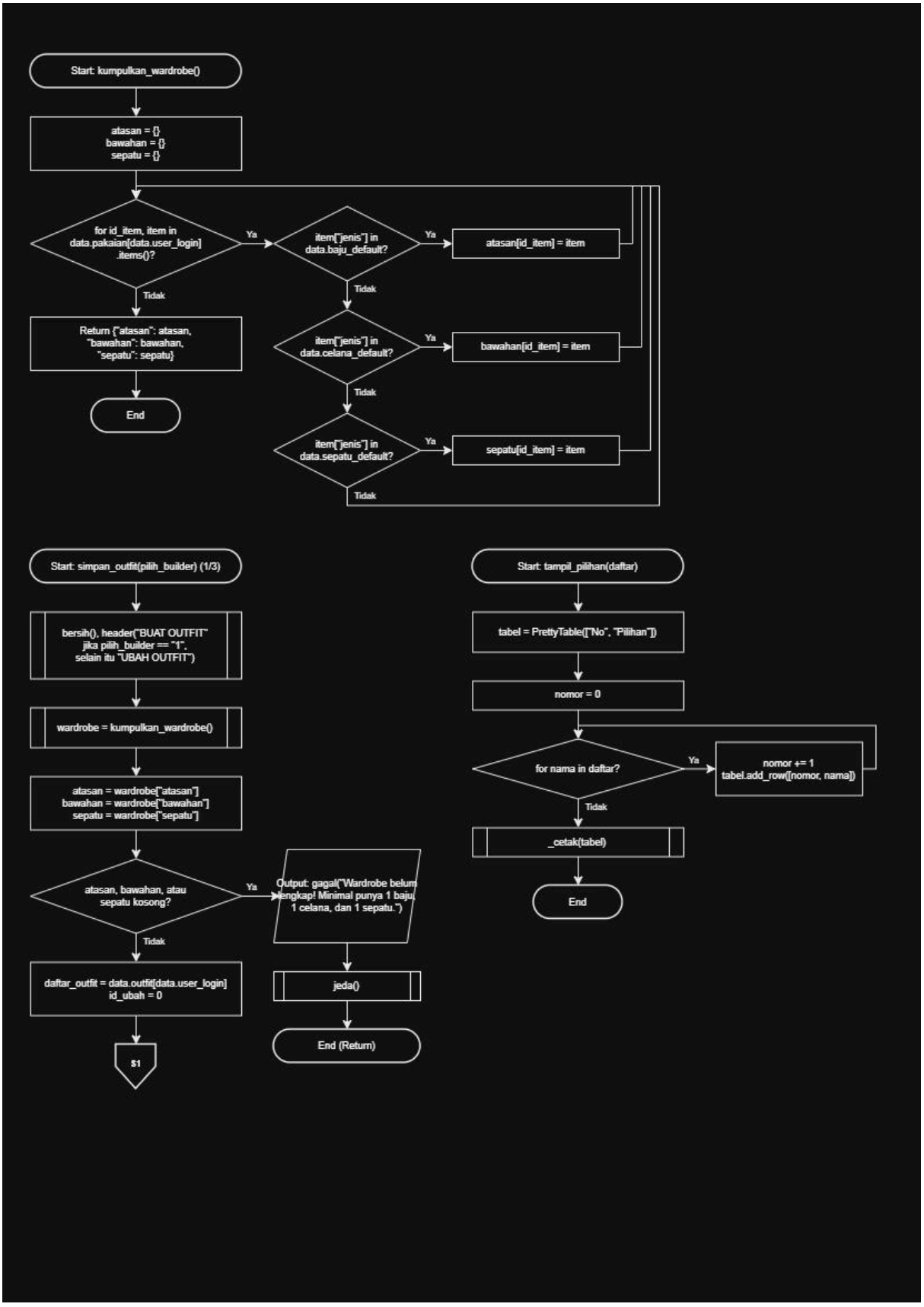
Gambar 1.5 Flowchart Halaman 5



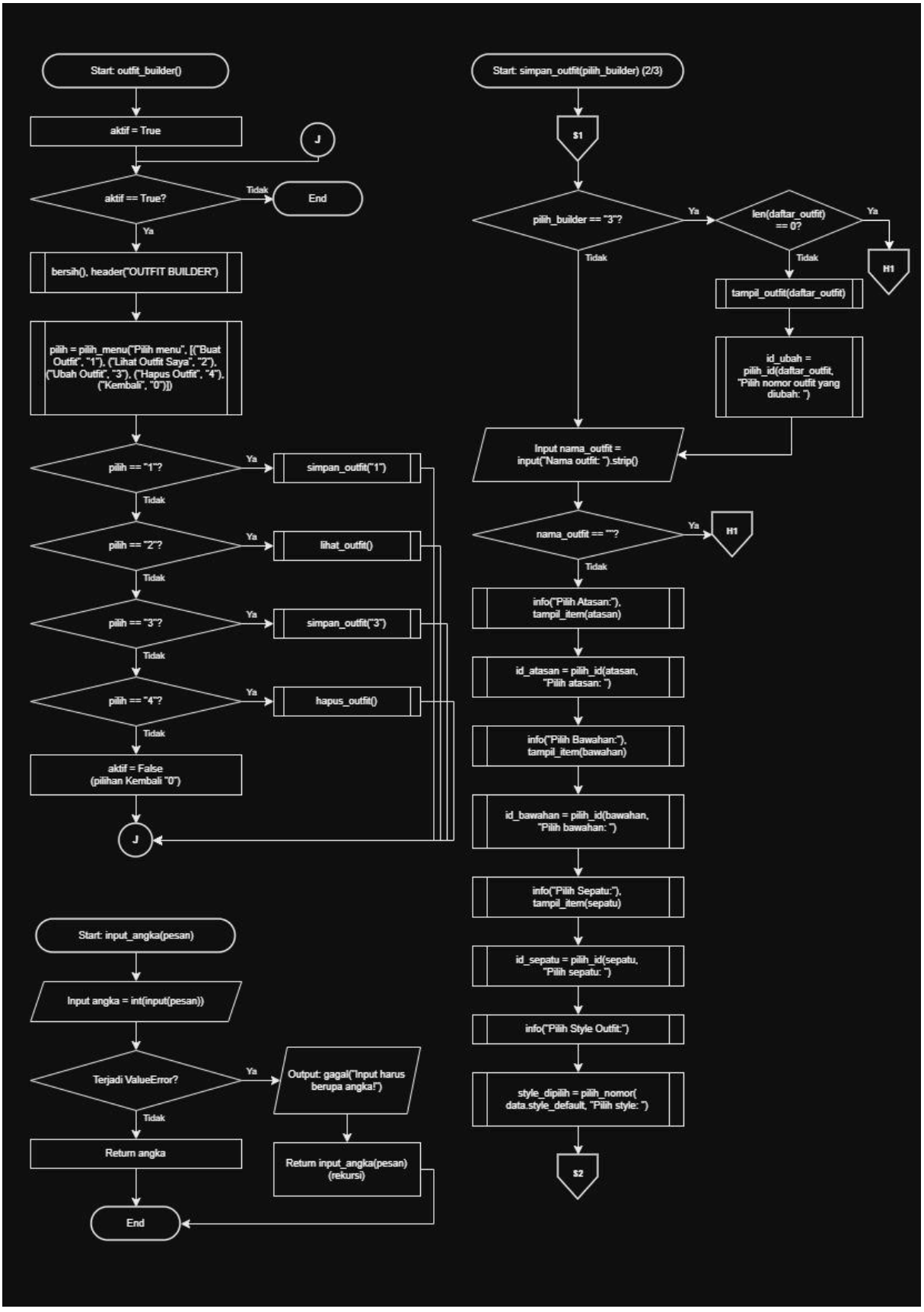
Gambar 1.6 Flowchart Halaman 6



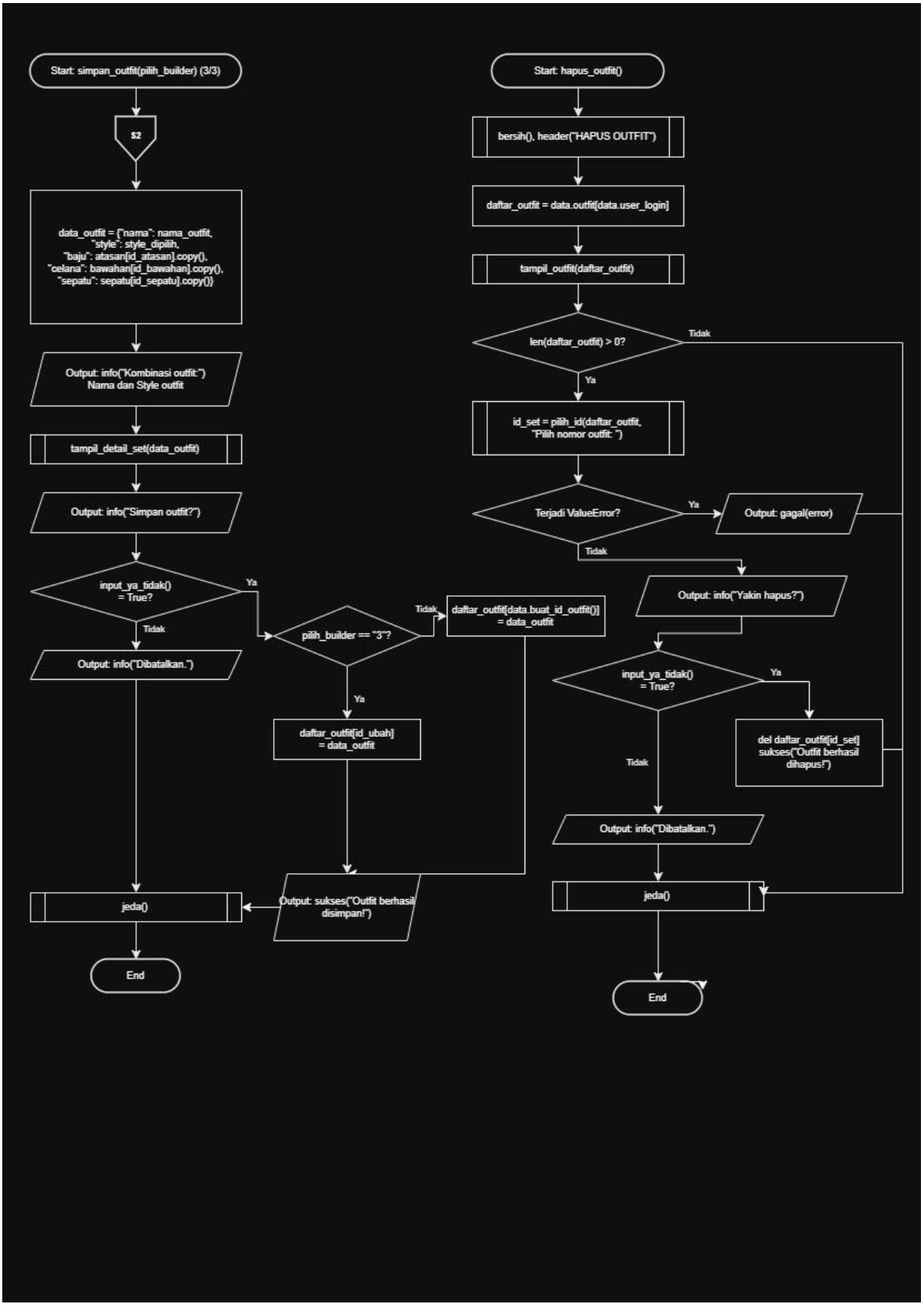
Gambar 1.7 Flowchart Halaman 7



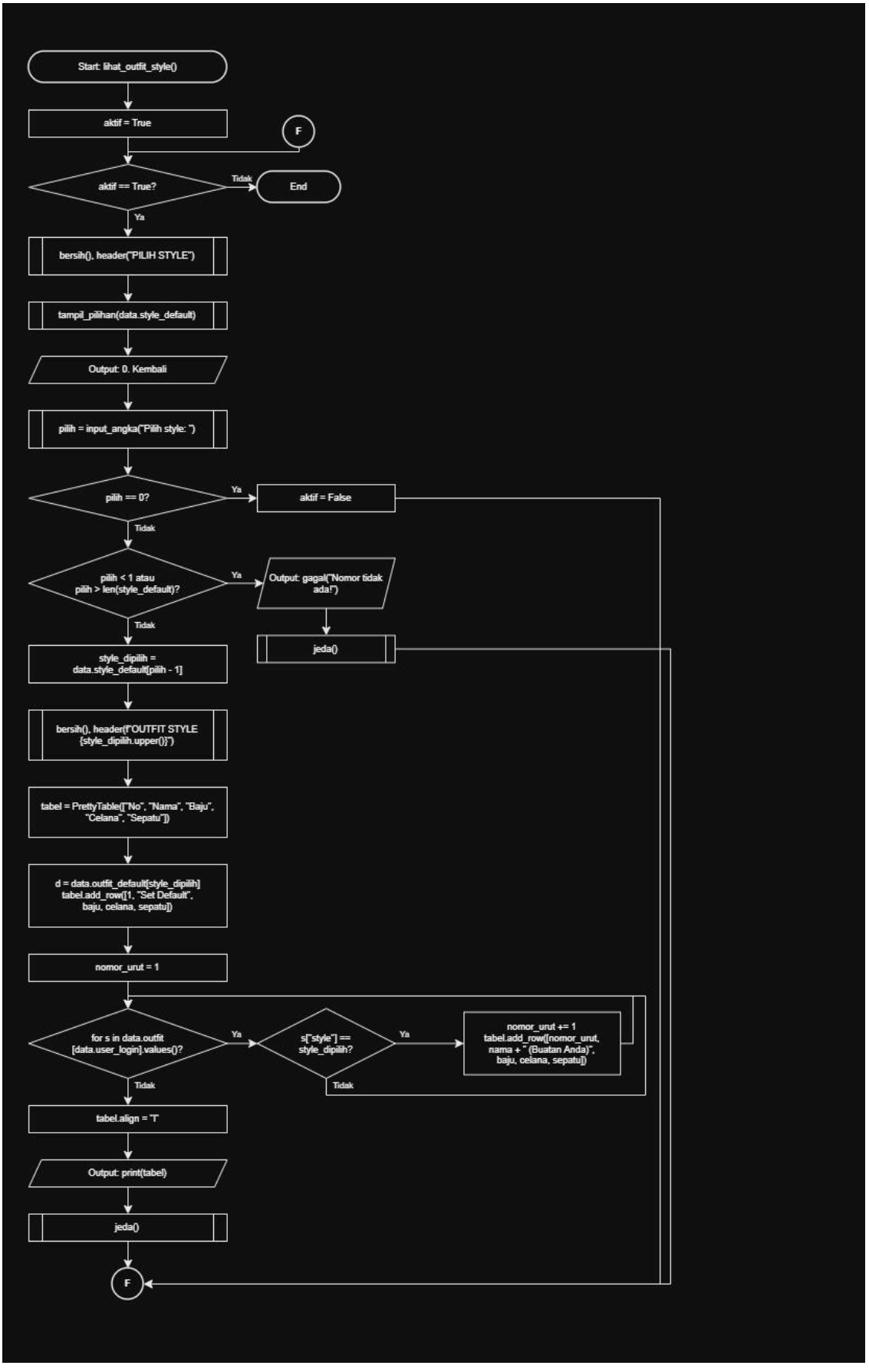
Gambar 1.8 Flowchart Halaman 8



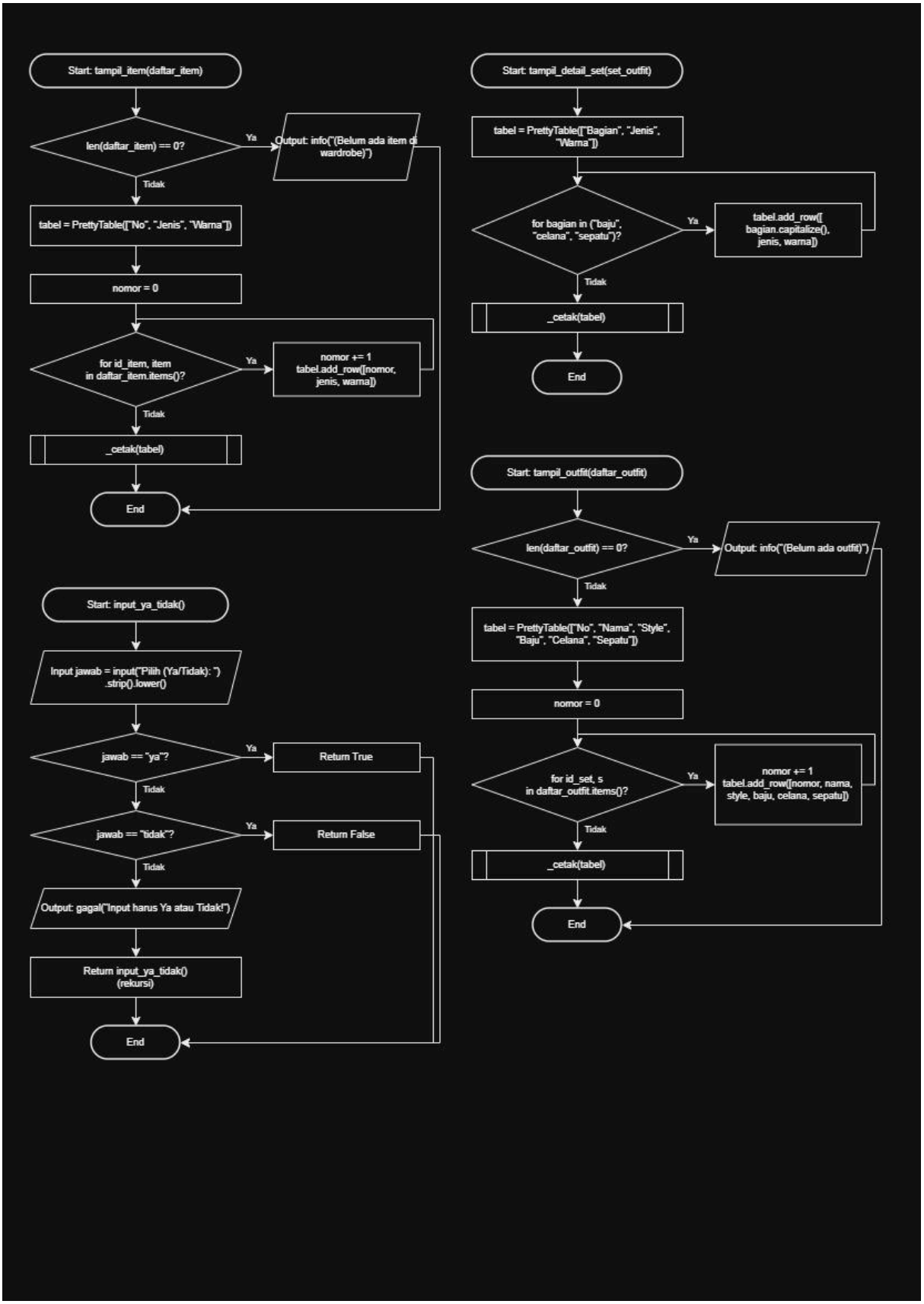
Gambar 1.9 Flowchart Halaman 9



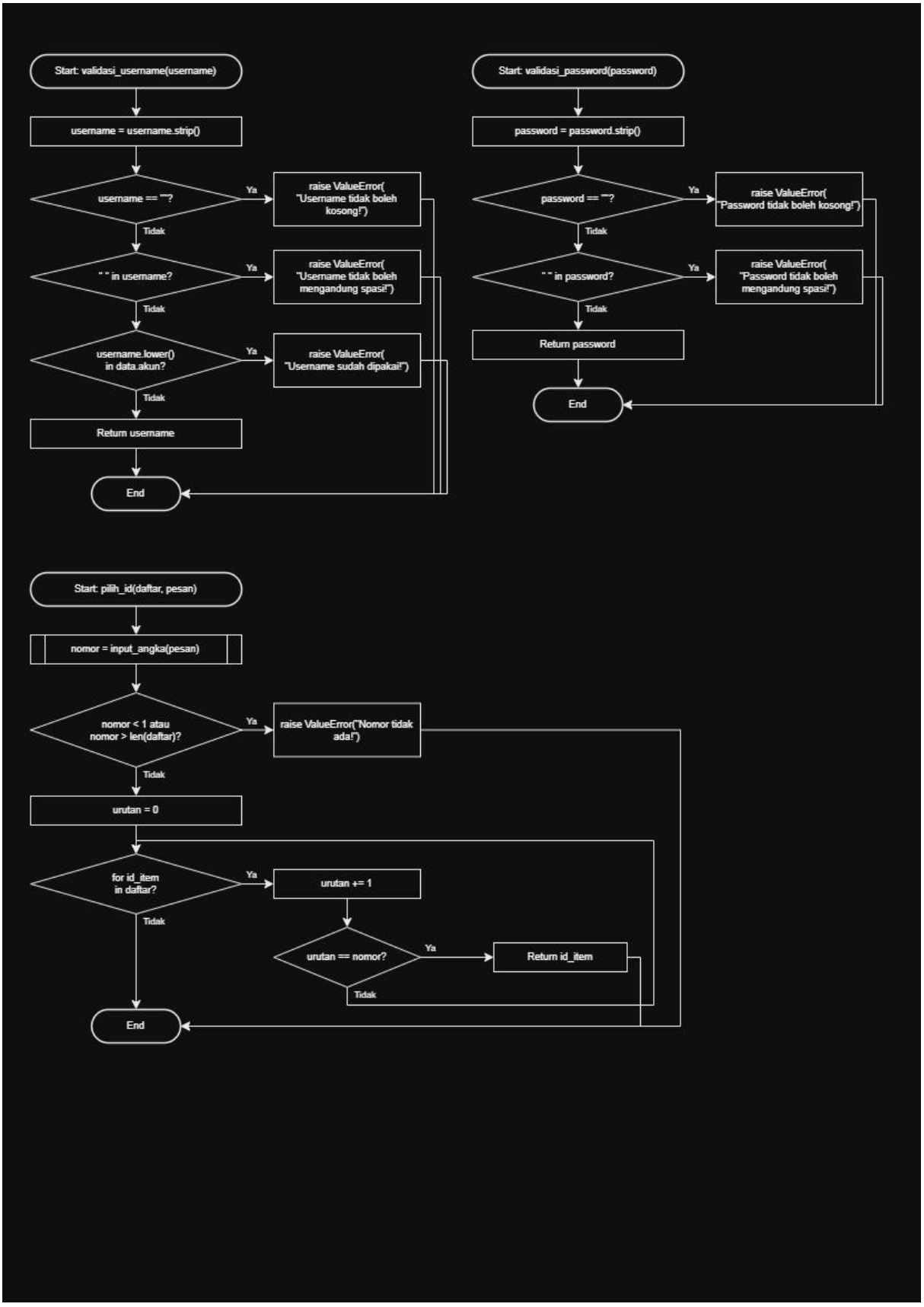
Gambar 1.10 Flowchart Halaman 10



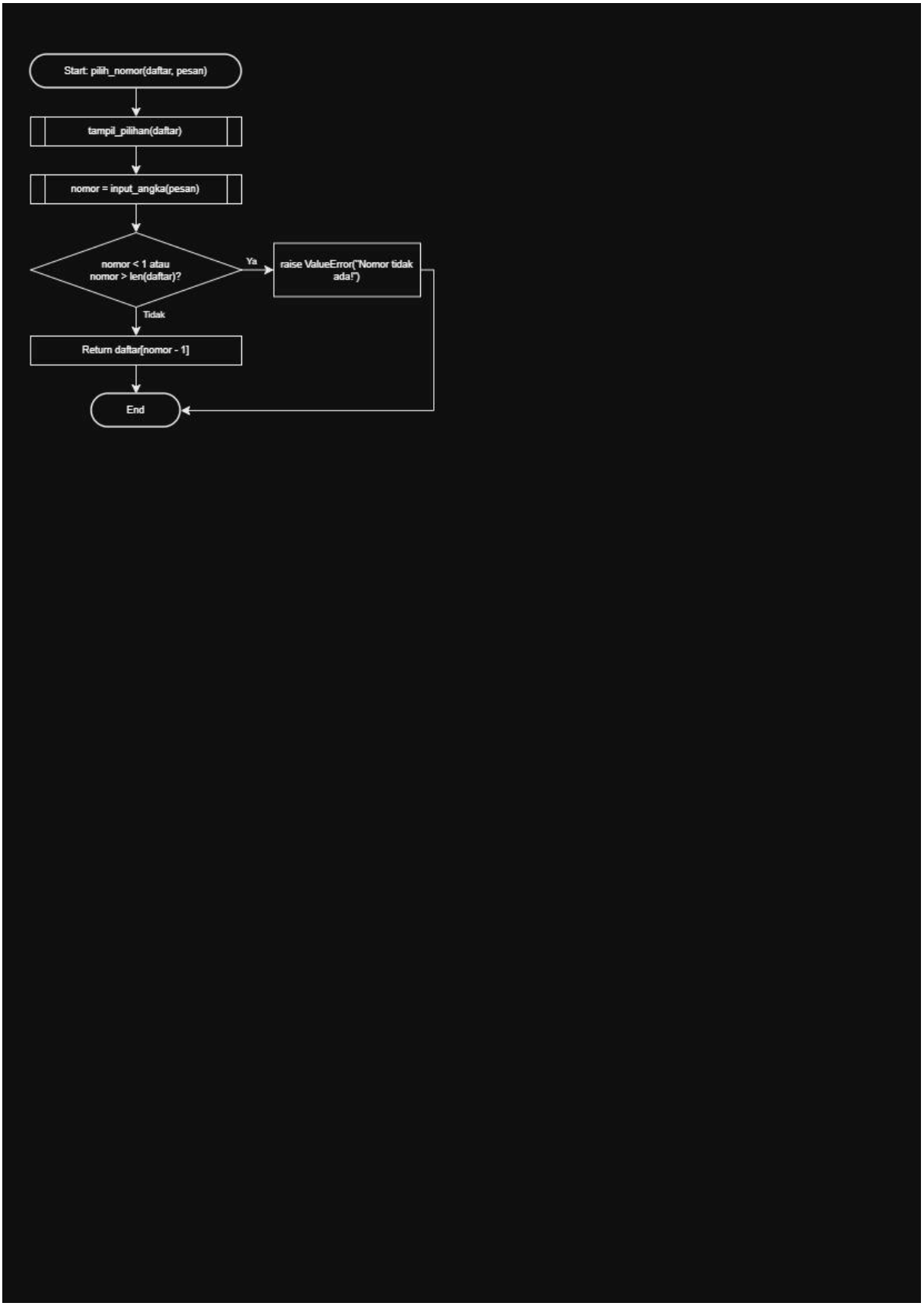
Gambar 1.11 Flowchart Halaman 11



Gambar 1.12 Flowchart Halaman 12



Gambar 1.13 Flowchart Halaman 13



Gambar 1.14 Flowchart Halaman 14

1.1 Penjelasan Flowchart

Flowchart ini menggambarkan alur program Manajemen & Perencanaan Outfit yang disusun berdasarkan fungsi dan prosedur pada setiap modul Python. Setiap fungsi dimulai dengan terminator Start: nama_fungsi() dan diakhiri dengan End atau Return. Pemanggilan fungsi dan prosedur menggunakan simbol predefined process, sedangkan percabangan kondisi menggunakan simbol decision. Blok try-except digambarkan melalui decision “Terjadi ValueError?” yang menentukan apakah program menampilkan pesan kesalahan atau melanjutkan proses.

Program dimulai dengan menginisialisasi variabel global berupa akun, pakaian, outfit, ID pakaian, ID outfit, user_login, dan role_login beserta data default. Selanjutnya, program menjalankan main() yang menampilkan menu Register, Login, dan Keluar secara berulang. Register menjalankan register() untuk membuat akun melalui validasi username dan password, sedangkan Login menjalankan login() dengan maksimal tiga kali percobaan. Jika login berhasil, sesi_login() akan mengarahkan pengguna berdasarkan role, yaitu Admin atau User. Setelah logout, data login dikosongkan dan pengguna kembali ke menu utama.

Admin diarahkan ke menu_admin() untuk melihat daftar akun, seluruh wardrobe, dan seluruh outfit pengguna. Sementara itu, User diarahkan ke menu_user() yang menyediakan tiga fitur utama, yaitu Kelola Wardrobe, Outfit Builder, dan Lihat Outfit Berdasarkan Style. Kelola Wardrobe menyediakan proses menambah, melihat, mengubah, dan menghapus item pakaian maupun sepatu. Penambahan dilakukan dengan memilih jenis dan warna pakaian, sedangkan perubahan dan penghapusan dilakukan dengan memilih item yang tersedia. Setiap item disimpan dalam dictionary menggunakan ID yang dibuat secara otomatis.

Outfit Builder dijalankan melalui outfit_builder() untuk membuat, melihat, mengubah, dan menghapus kombinasi outfit. Proses pembuatan dan perubahan dilakukan melalui simpan_outfit() yang memanggil kumpulkan_wardrobe() untuk mengelompokkan koleksi menjadi atasan, bawahan, dan sepatu. Jika seluruh kategori tersedia, pengguna memilih masing-masing item, menentukan nama dan style outfit, kemudian melakukan konfirmasi sebelum data disimpan atau diperbarui. Fitur Lihat Outfit Berdasarkan Style menjalankan lihat_outfit_style() untuk menampilkan kombinasi default dan outfit buatan pengguna berdasarkan kategori style yang dipilih.

Seluruh proses didukung oleh fungsi tambahan, seperti buat_id_pakaian() dan buat_id_outfit() untuk menghasilkan ID, pilih_nomor() dan pilih_id() untuk memvalidasi pilihan, serta input_angka() dan input_ya_tidak() yang menggunakan rekursi hingga input sesuai.

2. Deskripsi Singkat Program

Program ini merupakan sistem Manajemen dan Perencanaan Outfit berbasis terminal yang dirancang untuk mengelola koleksi pakaian pribadi dan merangkai kombinasi busana. Menu awal menyediakan tiga pilihan, yaitu Register, Login, dan Keluar. Pada Register, pengguna membuat akun dengan username dan password yang tidak boleh kosong atau mengandung spasi, serta username yang belum terdaftar (tanpa membedakan huruf besar dan kecil). Password yang diketik disembunyikan dan tampil sebagai tanda bintang. Pada Login, pengguna memiliki maksimal 3 kali percobaan sebelum dikembalikan ke menu awal. Sistem menyediakan satu akun admin bawaan, sedangkan akun hasil Register otomatis berperan sebagai user.

Setelah login, pengguna diarahkan ke menu sesuai perannya. Admin dapat melihat daftar seluruh akun, wardrobe semua pengguna, dan semua outfit yang tersimpan. User dapat memilih Kelola Wardrobe, Outfit Builder, Lihat Outfit Berdasarkan Style, atau Logout. Kelola Wardrobe digunakan untuk menambah item berdasarkan jenis dan warna, melihat koleksi, mengubah warna item, dan menghapus item dengan konfirmasi.

Outfit Builder memungkinkan pengguna membuat outfit apabila telah memiliki minimal 1 baju, 1 celana, dan 1 sepatu, dengan memasukkan nama, memilih komponen, dan menentukan style, lalu melihat, mengubah, atau menghapus outfit tersebut. Fitur Lihat Outfit Berdasarkan Style menampilkan set bawaan sistem dan outfit buatan pengguna sesuai style yang dipilih, yaitu Casual, Semi Casual, Semi Formal, atau Formal.

Pada Posttest 8, program dikembangkan dari Posttest 7 dengan memakai library PrettyTable sehingga seluruh menu, daftar item, daftar outfit, dan data admin tampil dalam bentuk tabel. Program juga dipecah menjadi 10 file berdasarkan fungsionalitasnya, yaitu main.py, data.py, tampilan.py, validasi.py, menu.py, autentikasi.py, admin.py, pengguna.py, wardrobe.py, dan outfit.py, yang saling terhubung dengan perintah import. Sebagai poin plus, program memakai library colorama untuk memberi warna pada pesan dan header, serta pwinput untuk menyembunyikan password saat diketik.

Kesalahan input ditangani dengan try-except (ValueError dan KeyError), raise ValueError, dan blok else. Fungsi rekursif input_angka() dan input_ya_tidak() memanggil dirinya sendiri sampai masukan valid.

3. Source Code

3.1 main.py

```
from tampilan import bersih, header, info
from menu import pilih_menu
from autentikasi import register, login, sesi_login


def main():
    jalan = True
    while jalan:
        bersih()
        header("MANAJEMEN & PERENCANAAN OUTFIT")
        pilih = pilih_menu("Silahkan Pilih Menu", [
            ("Register", "1"),
            ("Login", "2"),
            ("Keluar", "3"),
        ])
        if pilih == "1":
            register()
        elif pilih == "2":
            if login():
                sesi_login()
        else:
            info("Terima kasih!")
            jalan = False


if __name__ == "__main__":
    main()
```

3.2 data.py

```
akun = {
    "admin": {"username": "admin", "password": "admin123", "role": "admin"}
}
pakaian = {}
outfit = {}
id_pakaian = 1
id_outfit = 1
user_login = ""
role_login = ""

style_default = ("Casual", "Semi Casual", "Semi Formal", "Formal")
baju_default = ("Kemeja", "Polo Shirt", "Kaos")
celana_default = ("Celana Bahan", "Celana Chino", "Celana Jeans")
```

```
sepatu_default = ("Loafers", "Sneakers")
warna_default = ("Hitam", "Putih", "Navy", "Abu-abu", "Coklat")

outfit_default = {
    "Casual": {
        "baju": {"jenis": "Kaos", "warna": "Putih"},
        "celana": {"jenis": "Celana Jeans", "warna": "Navy"},
        "sepatu": {"jenis": "Sneakers", "warna": "Putih"},
    },
    "Semi Casual": {
        "baju": {"jenis": "Polo Shirt", "warna": "Navy"},
        "celana": {"jenis": "Celana Jeans", "warna": "Hitam"},
        "sepatu": {"jenis": "Sneakers", "warna": "Putih"},
    },
    "Semi Formal": {
        "baju": {"jenis": "Kemeja", "warna": "Putih"},
        "celana": {"jenis": "Celana Chino", "warna": "Coklat"},
        "sepatu": {"jenis": "Loafers", "warna": "Coklat"},
    },
    "Formal": {
        "baju": {"jenis": "Kemeja", "warna": "Putih"},
        "celana": {"jenis": "Celana Bahan", "warna": "Hitam"},
        "sepatu": {"jenis": "Loafers", "warna": "Hitam"},
    },
}


def buat_id_pakaian():
    global id_pakaian
    id_baru = id_pakaian
    id_pakaian += 1
    return id_baru


def buat_id_outfit():
    global id_outfit
    id_baru = id_outfit
    id_outfit += 1
    return id_baru
```

3.3 tampilan.py

```
import os
from colorama import Fore, init
from prettytable import PrettyTable

init(autoreset=True)
```

```
def bersih():
    os.system("cls" if os.name == "nt" else "clear")


def jeda():
    input("Tekan Enter...")


def header(judul):
    tabel = PrettyTable()
    tabel.header = False
    tabel.add_row([judul])
    tabel.align = "c"
    print(Fore.WHITE + str(tabel))


def sukses(pesan):
    print(Fore.GREEN + pesan)


def gagal(pesan):
    print(Fore.RED + str(pesan))


def info(pesan):
    print(Fore.YELLOW + pesan)


def _cetak(tabel):
    tabel.align = "l"
    print(tabel)


def tampil_pilihan(daftar):
    tabel = PrettyTable(["No", "Pilihan"])
    nomor = 0
    for nama in daftar:
        nomor += 1
        tabel.add_row([nomor, nama])
    _cetak(tabel)


def tampil_item(daftar_item):
    if len(daftar_item) == 0:
        info("(Belum ada item di wardrobe)")
        return
```

```
    tabel = PrettyTable(["No", "Jenis", "Warna"])
    nomor = 0
    for id_item, item in daftar_item.items():
        nomor += 1
        tabel.add_row([nomor, item["jenis"], item["warna"]])
    _cetak(tabel)


def tampil_detail_set(set_outfit):
    tabel = PrettyTable(["Bagian", "Jenis", "Warna"])
    for bagian in ("baju", "celana", "sepatu"):
        tabel.add_row([bagian.capitalize(), set_outfit[bagian]["jenis"],
set_outfit[bagian]["warna"]])
    _cetak(tabel)


def tampil_outfit(daftar_outfit):
    if len(daftar_outfit) == 0:
        info("(Belum ada outfit)")
        return
    tabel = PrettyTable(["No", "Nama", "Style", "Baju", "Celana", "Sepatu"])
    nomor = 0
    for id_set, s in daftar_outfit.items():
        nomor += 1
        tabel.add_row([
            nomor, s["nama"], s["style"],
            f"{s['baju']['jenis']} {s['baju']['warna']}",
            f"{s['celana']['jenis']} {s['celana']['warna']}",
            f"{s['sepatu']['jenis']} {s['sepatu']['warna']}",
        ])
    _cetak(tabel)
```

3.4 validasi.py

```
import data
from tampilan import tampil_pilihan, gagal


def input_ya_tidak():
    jawab = input("Pilih (Ya/Tidak): ").strip().lower()
    if jawab == "ya":
        return True
    if jawab == "tidak":
        return False
    gagal("Input harus Ya atau Tidak!")
    return input_ya_tidak()
```

```python
def input_angka(pesan):
    try:
        angka = int(input(pesan))
    except ValueError:
        gagal("Input harus berupa angka!")
        return input_angka(pesan)
    else:
        return angka


def validasi_username(username):
    username = username.strip()
    if username == "":
        raise ValueError("Username tidak boleh kosong!")
    if " " in username:
        raise ValueError("Username tidak boleh mengandung spasi!")
    if username.lower() in data.akun:
        raise ValueError("Username sudah dipakai!")
    return username


def validasi_password(password):
    password = password.strip()
    if password == "":
        raise ValueError("Password tidak boleh kosong!")
    if " " in password:
        raise ValueError("Password tidak boleh mengandung spasi!")
    return password


def pilih_nomor(daftar, pesan):
    tampil_pilihan(daftar)
    nomor = input_angka(pesan)
    if nomor < 1 or nomor > len(daftar):
        raise ValueError("Nomor tidak ada!")
    return daftar[nomor - 1]


def pilih_id(daftar, pesan):
    nomor = input_angka(pesan)
    if nomor < 1 or nomor > len(daftar):
        raise ValueError("Nomor tidak ada!")
    urutan = 0
    for id_item in daftar:
        urutan += 1
        if urutan == nomor:
            return id_item
```

3.5 menu.py

```
from prettytable import PrettyTable
from tampilan import gagal


def pilih_menu(judul, pilihan):
    tabel = PrettyTable(["Kode", "Menu"])
    daftar_kode = []
    for label, kode in pilihan:
        tabel.add_row([kode, label])
        daftar_kode.append(kode)
    tabel.align = "l"
    print(tabel)

    while True:
        jawab = input(f"{judul}: ").strip()
        if jawab in daftar_kode:
            return jawab
        gagal("Pilihan tidak valid!")
```

3.6 autentikasi.py

```
import pwinput
import data
from tampilan import bersih, header, jeda, sukses, gagal
from validasi import validasi_username, validasi_password
from admin import menu_admin
from pengguna import menu_user


def register():
    bersih()
    header("REGISTER")
    try:
        username = validasi_username(input("Username: "))
        password = validasi_password(pwinput.pwinput("Password: "))
    except ValueError as error:
        gagal(error)
    else:
        kunci = username.lower()
        data.akun[kunci] = {"username": username, "password": password,
"role": "user"}
        data.pakaian[kunci] = {}
        data.outfit[kunci] = {}
        sukses("Register berhasil! Silakan login.")
    jeda()
```

```
def login():
    bersih()
    header("LOGIN")
    for percobaan in range(3):
        username = input("Username: ").strip()
        password = pwinput.pwinput("Password: ").strip()
        kunci = username.lower()
        try:
            akun_ada = data.akun[kunci]
        except KeyError:
            gagal(f"Username atau password salah! Sisa percobaan: {2 -
percobaan}")
            continue
        if akun_ada["password"] == password:
            data.user_login = kunci
            data.role_login = akun_ada["role"]
            return True
        gagal(f"Username atau password salah! Sisa percobaan: {2 -
percobaan}")
    gagal("Login gagal 3 kali, kembali ke menu awal.")
    jeda()
    return False


def sesi_login():
    if data.role_login == "admin":
        menu_admin()
    else:
        menu_user()
    data.user_login = ""
    data.role_login = ""
```

3.7 admin.py

```
from prettytable import PrettyTable
import data
from tampilan import bersih, header, jeda, info
from menu import pilih_menu


def lihat_akun():
    bersih()
    header("DAFTAR AKUN TERDAFTAR")
    tabel = PrettyTable(["No", "Username", "Role"])
    nomor = 0
    for kunci, akun_ada in data.akun.items():
        nomor += 1
```

```
        tabel.add_row([nomor, akun_ada["username"], akun_ada["role"]])
    tabel.align = "l"
    print(tabel)
    jeda()


def lihat_semua_wardrobe():
    bersih()
    header("SEMUA WARDROBE USER")
    tabel = PrettyTable(["No", "User", "Jenis", "Warna"])
    nomor = 0
    for pemilik, daftar in data.pakaian.items():
        for item in daftar.values():
            nomor += 1
            tabel.add_row([nomor, data.akun[pemilik]["username"],
item["jenis"], item["warna"]])
    if nomor == 0:
        info("(Belum ada data wardrobe)")
    else:
        tabel.align = "l"
        print(tabel)
    jeda()


def lihat_semua_outfit():
    bersih()
    header("SEMUA OUTFIT")
    tabel = PrettyTable(["No", "User", "Nama Outfit", "Style"])
    nomor = 0
    for pemilik, daftar in data.outfit.items():
        for s in daftar.values():
            nomor += 1
            tabel.add_row([nomor, data.akun[pemilik]["username"], s["nama"],
s["style"]])
    if nomor == 0:
        info("(Belum ada data outfit)")
    else:
        tabel.align = "l"
        print(tabel)
    jeda()


def menu_admin():
    aktif = True
    while aktif:
        bersih()
        header("MENU ADMIN")
        pilih = pilih_menu("Pilih menu", [
```

```
            ("Lihat Daftar Akun", "1"),
            ("Lihat Semua Wardrobe User", "2"),
            ("Lihat Semua Outfit User", "3"),
            ("Logout", "0"),
        ])
        if pilih == "1":
            lihat_akun()
        elif pilih == "2":
            lihat_semua_wardrobe()
        elif pilih == "3":
            lihat_semua_outfit()
        else:
            aktif = False
```

3.8 pengguna.py

```
import data
from tampilan import bersih, header
from menu import pilih_menu
from wardrobe import kelola_wardrobe
from outfit import outfit_builder, lihat_outfit_style


def menu_user():
    aktif = True
    while aktif:
        bersih()
        header("MENU PENGGUNA")
        print("Halo,", data.akun[data.user_login]["username"])
        pilih = pilih_menu("Pilih menu", [
            ("Kelola Wardrobe (Pakaian & Sepatu)", "1"),
            ("Outfit Builder", "2"),
            ("Lihat Outfit Berdasarkan Style", "3"),
            ("Logout", "0"),
        ])
        if pilih == "1":
            kelola_wardrobe()
        elif pilih == "2":
            outfit_builder()
        elif pilih == "3":
            lihat_outfit_style()
        else:
            aktif = False
```

3.9 wardrobe.py

```
import data
from tampilan import bersih, header, jeda, tampil_item, sukses, gagal, info
from validasi import pilih_nomor, pilih_id, input_ya_tidak
from menu import pilih_menu


def kumpulkan_wardrobe():
    atasan = {}
    bawahan = {}
    sepatu = {}
    for id_item, item in data.pakaian[data.user_login].items():
        if item["jenis"] in data.baju_default:
            atasan[id_item] = item
        elif item["jenis"] in data.celana_default:
            bawahan[id_item] = item
        elif item["jenis"] in data.sepatu_default:
            sepatu[id_item] = item
    return {"atasan": atasan, "bawahan": bawahan, "sepatu": sepatu}


def tambah_item():
    bersih()
    header("TAMBAH ITEM")
    daftar_jenis = data.baju_default + data.celana_default +
data.sepatu_default
    try:
        jenis = pilih_nomor(daftar_jenis, "Pilih jenis: ")
        warna = pilih_nomor(data.warna_default, "Pilih warna: ")
    except ValueError as error:
        gagal(error)
    else:
        data.pakaian[data.user_login][data.buat_id_pakaian()] = {"jenis":
jenis, "warna": warna}
        sukses("Item berhasil ditambahkan!")
    jeda()


def lihat_wardrobe():
    bersih()
    header("WARDROBE ANDA")
    tampil_item(data.pakaian[data.user_login])
    jeda()


def ubah_item():
    bersih()
    header("UBAH ITEM")
```

```
    daftar_pakaian = data.pakaian[data.user_login]
    tampil_item(daftar_pakaian)
    if len(daftar_pakaian) > 0:
        try:
            id_item = pilih_id(daftar_pakaian, "Pilih nomor item: ")
            warna = pilih_nomor(data.warna_default, "Pilih warna baru: ")
        except ValueError as error:
            gagal(error)
        else:
            daftar_pakaian[id_item]["warna"] = warna
            sukses("Item berhasil diubah!")
    jeda()


def hapus_item():
    bersih()
    header("HAPUS ITEM")
    daftar_pakaian = data.pakaian[data.user_login]
    tampil_item(daftar_pakaian)
    if len(daftar_pakaian) > 0:
        try:
            id_item = pilih_id(daftar_pakaian, "Pilih nomor item: ")
        except ValueError as error:
            gagal(error)
        else:
            info("Yakin hapus?")
            if input_ya_tidak():
                del daftar_pakaian[id_item]
                sukses("Item berhasil dihapus!")
            else:
                info("Dibatalkan.")
    jeda()


def kelola_wardrobe():
    aktif = True
    while aktif:
        bersih()
        header("KELOLA WARDROBE")
        pilih = pilih_menu("Pilih menu", [
            ("Tambah Item", "1"),
            ("Lihat Wardrobe", "2"),
            ("Ubah Item", "3"),
            ("Hapus Item", "4"),
            ("Kembali", "0"),
        ])
        if pilih == "1":
            tambah_item()
```

```
        elif pilih == "2":
            lihat_wardrobe()
        elif pilih == "3":
            ubah_item()
        elif pilih == "4":
            hapus_item()
        else:
            aktif = False
```

3.10 outfit.py

```
from prettytable import PrettyTable
import data
from tampilan import (bersih, header, jeda, tampil_item, tampil_outfit,
tampil_detail_set,
                      tampil_pilihan, sukses, gagal, info)
from validasi import pilih_nomor, pilih_id, input_ya_tidak, input_angka
from wardrobe import kumpulkan_wardrobe
from menu import pilih_menu


def simpan_outfit(pilih_builder):
    bersih()
    header("BUAT OUTFIT" if pilih_builder == "1" else "UBAH OUTFIT")
    wardrobe = kumpulkan_wardrobe()
    atasan = wardrobe["atasan"]
    bawahan = wardrobe["bawahan"]
    sepatu = wardrobe["sepatu"]

    if len(atasan) == 0 or len(bawahan) == 0 or len(sepatu) == 0:
        gagal("Wardrobe belum lengkap!")
        gagal("Minimal punya 1 baju, 1 celana, dan 1 sepatu.")
        gagal("Tambahkan dulu di menu Kelola Wardrobe.")
        jeda()
        return

    daftar_outfit = data.outfit[data.user_login]
    id_ubah = 0

    try:
        if pilih_builder == "3":
            if len(daftar_outfit) == 0:
                raise ValueError("(Belum ada outfit)")
            tampil_outfit(daftar_outfit)
            id_ubah = pilih_id(daftar_outfit, "Pilih nomor outfit yang diubah:
")

        nama_outfit = input("Nama outfit: ").strip()
```

```
        if nama_outfit == "":
            raise ValueError("Nama outfit tidak boleh kosong!")

        info("Pilih Atasan:")
        tampil_item(atasan)
        id_atasan = pilih_id(atasan, "Pilih atasan: ")

        info("Pilih Bawahan:")
        tampil_item(bawahan)
        id_bawahan = pilih_id(bawahan, "Pilih bawahan: ")

        info("Pilih Sepatu:")
        tampil_item(sepatu)
        id_sepatu = pilih_id(sepatu, "Pilih sepatu: ")

        info("Pilih Style Outfit:")
        style_dipilih = pilih_nomor(data.style_default, "Pilih style: ")
    except ValueError as error:
        gagal(error)
        jeda()
        return

    data_outfit = {
        "nama": nama_outfit,
        "style": style_dipilih,
        "baju": atasan[id_atasan].copy(),
        "celana": bawahan[id_bawahan].copy(),
        "sepatu": sepatu[id_sepatu].copy(),
    }

    info("Kombinasi outfit:")
    print(f"Nama  : {nama_outfit}")
    print(f"Style : {style_dipilih}")
    tampil_detail_set(data_outfit)
    info("Simpan outfit?")

    if input_ya_tidak():
        if pilih_builder == "3":
            daftar_outfit[id_ubah] = data_outfit
        else:
            daftar_outfit[data.buat_id_outfit()] = data_outfit
        sukses("Outfit berhasil disimpan!")
    else:
        info("Dibatalkan.")
    jeda()


def lihat_outfit():
```

```
    bersih()
    header("OUTFIT SAYA")
    tampil_outfit(data.outfit[data.user_login])
    jeda()


def hapus_outfit():
    bersih()
    header("HAPUS OUTFIT")
    daftar_outfit = data.outfit[data.user_login]
    tampil_outfit(daftar_outfit)
    if len(daftar_outfit) > 0:
        try:
            id_set = pilih_id(daftar_outfit, "Pilih nomor outfit: ")
        except ValueError as error:
            gagal(error)
        else:
            info("Yakin hapus?")
            if input_ya_tidak():
                del daftar_outfit[id_set]
                sukses("Outfit berhasil dihapus!")
            else:
                info("Dibatalkan.")
    jeda()


def outfit_builder():
    aktif = True
    while aktif:
        bersih()
        header("OUTFIT BUILDER")
        pilih = pilih_menu("Pilih menu", [
            ("Buat Outfit", "1"),
            ("Lihat Outfit Saya", "2"),
            ("Ubah Outfit", "3"),
            ("Hapus Outfit", "4"),
            ("Kembali", "0"),
        ])
        if pilih == "1":
            simpan_outfit("1")
        elif pilih == "2":
            lihat_outfit()
        elif pilih == "3":
            simpan_outfit("3")
        elif pilih == "4":
            hapus_outfit()
        else:
            aktif = False
```

```python
def lihat_outfit_style():
    aktif = True
    while aktif:
        bersih()
        header("PILIH STYLE")
        tampil_pilihan(data.style_default)
        print("0. Kembali")
        pilih = input_angka("Pilih style: ")

        if pilih == 0:
            aktif = False
        elif pilih < 1 or pilih > len(data.style_default):
            gagal("Nomor tidak ada!")
            jeda()
        else:
            style_dipilih = data.style_default[pilih - 1]
            bersih()
            header(f"OUTFIT STYLE {style_dipilih.upper()}")
            tabel = PrettyTable(["No", "Nama", "Baju", "Celana", "Sepatu"])
            d = data.outfit_default[style_dipilih]
            tabel.add_row([
                1, "Set Default",
                f"{d['baju']['jenis']} {d['baju']['warna']}",
                f"{d['celana']['jenis']} {d['celana']['warna']}",
                f"{d['sepatu']['jenis']} {d['sepatu']['warna']}",
            ])
            nomor_urut = 1
            for s in data.outfit[data.user_login].values():
                if s["style"] == style_dipilih:
                    nomor_urut += 1
                    tabel.add_row([
                        nomor_urut, f"{s['nama']} (Buatan Anda)",
                        f"{s['baju']['jenis']} {s['baju']['warna']}",
                        f"{s['celana']['jenis']} {s['celana']['warna']}",
                        f"{s['sepatu']['jenis']} {s['sepatu']['warna']}",
                    ])
            tabel.align = "l"
            print(tabel)
            jeda()
```

A. Program Utama

Program dipecah menjadi 10 file dalam satu folder dan hanya main.py yang dijalankan. main() menampilkan menu awal (Register, Login, Keluar) dalam perulangan. Jika login berhasil, sesi_login() dipanggil. Library eksternal yang dipakai adalah prettytable, colorama, dan pwinput.

Source Code:

```
from tampilan import bersih, header, info
from menu import pilih_menu
from autentikasi import register, login, sesi_login


def main():
    jalan = True
    while jalan:
        bersih()
        header("MANAJEMEN & PERENCANAAN OUTFIT")
        pilih = pilih_menu("Silahkan Pilih Menu", [
            ("Register", "1"),
            ("Login", "2"),
            ("Keluar", "3"),
        ])
        if pilih == "1":
            register()
        elif pilih == "2":
            if login():
                sesi_login()
        else:
            info("Terima kasih!")
            jalan = False


if __name__ == "__main__":
    main()
```

B. Modul Data

Semua data disimpan di data.py dan diakses file lain lewat data.nama_variabel. Isinya dictionary akun, pakaian, dan outfit, penghitung ID, data login, tuple pilihan, dan outfit_default. global hanya dipakai di dua fungsi pembuat ID.

Source Code:

```
akun = {
    "admin": {"username": "admin", "password": "admin123", "role": "admin"}
}
pakaian = {}
outfit = {}
id_pakaian = 1
id_outfit = 1
user_login = ""
role_login = ""

style_default = ("Casual", "Semi Casual", "Semi Formal", "Formal")
baju_default = ("Kemeja", "Polo Shirt", "Kaos")
celana_default = ("Celana Bahan", "Celana Chino", "Celana Jeans")
sepatu_default = ("Loafers", "Sneakers")
warna_default = ("Hitam", "Putih", "Navy", "Abu-abu", "Coklat")

outfit_default = {
    "Casual": {
        "baju": {"jenis": "Kaos", "warna": "Putih"},
        "celana": {"jenis": "Celana Jeans", "warna": "Navy"},
        "sepatu": {"jenis": "Sneakers", "warna": "Putih"},
    },
    "Semi Casual": {
        "baju": {"jenis": "Polo Shirt", "warna": "Navy"},
        "celana": {"jenis": "Celana Jeans", "warna": "Hitam"},
        "sepatu": {"jenis": "Sneakers", "warna": "Putih"},
    },
    "Semi Formal": {
        "baju": {"jenis": "Kemeja", "warna": "Putih"},
        "celana": {"jenis": "Celana Chino", "warna": "Coklat"},
        "sepatu": {"jenis": "Loafers", "warna": "Coklat"},
    },
    "Formal": {
        "baju": {"jenis": "Kemeja", "warna": "Putih"},
        "celana": {"jenis": "Celana Bahan", "warna": "Hitam"},
        "sepatu": {"jenis": "Loafers", "warna": "Hitam"},
    },
}
```

```
def buat_id_pakaian():
    global id_pakaian
    id_baru = id_pakaian
    id_pakaian += 1
    return id_baru


def buat_id_outfit():
    global id_outfit
    id_baru = id_outfit
    id_outfit += 1
    return id_baru
```

C. Prosedur Tampilan

tampilan.py mengatur tampilan terminal: membersihkan layar, header, pesan berwarna (hijau, merah, kuning), dan tabel data dengan nomor urut. Semua berupa prosedur tanpa return.

Source Code:

```
import os
from colorama import Fore, init
from prettytable import PrettyTable

init(autoreset=True)


def bersih():
    os.system("cls" if os.name == "nt" else "clear")


def jeda():
    input("Tekan Enter...")


def header(judul):
    tabel = PrettyTable()
    tabel.header = False
    tabel.add_row([judul])
    tabel.align = "c"
    print(Fore.WHITE + str(tabel))


def sukses(pesan):
    print(Fore.GREEN + pesan)
```

```
def gagal(pesan):
    print(Fore.RED + str(pesan))


def info(pesan):
    print(Fore.YELLOW + pesan)


def _cetak(tabel):
    tabel.align = "l"
    print(tabel)


def tampil_pilihan(daftar):
    tabel = PrettyTable(["No", "Pilihan"])
    nomor = 0
    for nama in daftar:
        nomor += 1
        tabel.add_row([nomor, nama])
    _cetak(tabel)


def tampil_item(daftar_item):
    if len(daftar_item) == 0:
        info("(Belum ada item di wardrobe)")
        return
    tabel = PrettyTable(["No", "Jenis", "Warna"])
    nomor = 0
    for id_item, item in daftar_item.items():
        nomor += 1
        tabel.add_row([nomor, item["jenis"], item["warna"]])
    _cetak(tabel)


def tampil_detail_set(set_outfit):
    tabel = PrettyTable(["Bagian", "Jenis", "Warna"])
    for bagian in ("baju", "celana", "sepatu"):
        tabel.add_row([bagian.capitalize(), set_outfit[bagian]["jenis"],
set_outfit[bagian]["warna"]])
    _cetak(tabel)


def tampil_outfit(daftar_outfit):
    if len(daftar_outfit) == 0:
        info("(Belum ada outfit)")
        return
    tabel = PrettyTable(["No", "Nama", "Style", "Baju", "Celana", "Sepatu"])
```

```
    nomor = 0
    for id_set, s in daftar_outfit.items():
        nomor += 1
        tabel.add_row([
            nomor, s["nama"], s["style"],
            f"{s['baju']['jenis']} {s['baju']['warna']}",
            f"{s['celana']['jenis']} {s['celana']['warna']}",
            f"{s['sepatu']['jenis']} {s['sepatu']['warna']}",
        ])
    _cetak(tabel)
```

D. Fitur Register dan Login

register() memvalidasi username dan password dalam try-except-else, lalu menyimpan akun baru dengan role “user”. login() memberi tiga kali percobaan dan mengembalikan True atau False. sesi_login() mengarahkan ke menu admin atau user sesuai role, lalu mengosongkan data login saat selesai.

Source Code:

```
import pwinput
import data
from tampilan import bersih, header, jeda, sukses, gagal
from validasi import validasi_username, validasi_password
from admin import menu_admin
from pengguna import menu_user


def register():
    bersih()
    header("REGISTER")
    try:
        username = validasi_username(input("Username: "))
        password = validasi_password(pwinput.pwinput("Password: "))
    except ValueError as error:
        gagal(error)
    else:
        kunci = username.lower()
        data.akun[kunci] = {"username": username, "password": password,
"role": "user"}
        data.pakaian[kunci] = {}
        data.outfit[kunci] = {}
        sukses("Register berhasil! Silakan login.")
    jeda()
```

```
def login():
    bersih()
    header("LOGIN")
    for percobaan in range(3):
        username = input("Username: ").strip()
        password = pwinput.pwinput("Password: ").strip()
        kunci = username.lower()
        try:
            akun_ada = data.akun[kunci]
        except KeyError:
            gagal(f"Username atau password salah! Sisa percobaan: {2 -
percobaan}")
            continue
        if akun_ada["password"] == password:
            data.user_login = kunci
            data.role_login = akun_ada["role"]
            return True
        gagal(f"Username atau password salah! Sisa percobaan: {2 -
percobaan}")
    gagal("Login gagal 3 kali, kembali ke menu awal.")
    jeda()
    return False


def sesi_login():
    if data.role_login == "admin":
        menu_admin()
    else:
        menu_user()
    data.user_login = ""
    data.role_login = ""
```

E. Fitur Menu Admin

Admin dapat melihat daftar akun, wardrobe semua user, dan outfit semua user dalam bentuk tabel, dengan pesan khusus jika datanya kosong. menu_admin() berjalan dalam perulangan while sampai admin logout.

Source Code:

```
from prettytable import PrettyTable
import data
from tampilan import bersih, header, jeda, info
from menu import pilih_menu


def lihat_akun():
    bersih()
    header("DAFTAR AKUN TERDAFTAR")
    tabel = PrettyTable(["No", "Username", "Role"])
    nomor = 0
    for kunci, akun_ada in data.akun.items():
        nomor += 1
        tabel.add_row([nomor, akun_ada["username"], akun_ada["role"]])
    tabel.align = "l"
    print(tabel)
    jeda()


def lihat_semua_wardrobe():
    bersih()
    header("SEMUA WARDROBE USER")
    tabel = PrettyTable(["No", "User", "Jenis", "Warna"])
    nomor = 0
    for pemilik, daftar in data.pakaian.items():
        for item in daftar.values():
            nomor += 1
            tabel.add_row([nomor, data.akun[pemilik]["username"], item["jenis"],
item["warna"]])
    if nomor == 0:
        info("(Belum ada data wardrobe)")
    else:
        tabel.align = "l"
        print(tabel)
    jeda()


def lihat_semua_outfit():
    bersih()
```

```
    header("SEMUA OUTFIT")
    tabel = PrettyTable(["No", "User", "Nama Outfit", "Style"])
    nomor = 0
    for pemilik, daftar in data.outfit.items():
        for s in daftar.values():
            nomor += 1
            tabel.add_row([nomor, data.akun[pemilik]["username"], s["nama"],
s["style"]])
    if nomor == 0:
        info("(Belum ada data outfit)")
    else:
        tabel.align = "l"
        print(tabel)
    jeda()


def menu_admin():
    aktif = True
    while aktif:
        bersih()
        header("MENU ADMIN")
        pilih = pilih_menu("Pilih menu", [
            ("Lihat Daftar Akun", "1"),
            ("Lihat Semua Wardrobe User", "2"),
            ("Lihat Semua Outfit User", "3"),
            ("Logout", "0"),
        ])
        if pilih == "1":
            lihat_akun()
        elif pilih == "2":
            lihat_semua_wardrobe()
        elif pilih == "3":
            lihat_semua_outfit()
        else:
            aktif = False
```

F. Menu Pengguna

menu_user() menampilkan tiga fitur utama (Kelola Wardrobe, Outfit Builder, Lihat Outfit Berdasarkan Style) dan logout, lalu memanggil fungsi yang sesuai dalam perulangan while.

Source Code:

```
import data
from tampilan import bersih, header
from menu import pilih_menu
from wardrobe import kelola_wardrobe
from outfit import outfit_builder, lihat_outfit_style


def menu_user():
    aktif = True
    while aktif:
        bersih()
        header("MENU PENGGUNA")
        print("Halo,", data.akun[data.user_login]["username"])
        pilih = pilih_menu("Pilih menu", [
            ("Kelola Wardrobe (Pakaian & Sepatu)", "1"),
            ("Outfit Builder", "2"),
            ("Lihat Outfit Berdasarkan Style", "3"),
            ("Logout", "0"),
        ])
        if pilih == "1":
            kelola_wardrobe()
        elif pilih == "2":
            outfit_builder()
        elif pilih == "3":
            lihat_outfit_style()
        else:
            aktif = False
```

G. Fungsi Menu

pilih_menu() menampilkan daftar menu dalam tabel berisi kode dan nama menu. User mengetik kodenya, dan perulangan while True terus meminta input sampai kode yang dimasukkan valid.

Source Code:

```
from prettytable import PrettyTable
from tampilan import gagal


def pilih_menu(judul, pilihan):
    tabel = PrettyTable(["Kode", "Menu"])
    daftar_kode = []
    for label, kode in pilihan:
        tabel.add_row([kode, label])
        daftar_kode.append(kode)
    tabel.align = "l"
    print(tabel)

    while True:
        jawab = input(f"{judul}: ").strip()
        if jawab in daftar_kode:
            return jawab
        gagal("Pilihan tidak valid!")
```

H. Fitur Kelola Wardrobe

Ada empat operasi: tambah, lihat, ubah warna, dan hapus item. Semuanya hanya mengakses wardrobe milik user yang login, error ditangkap dengan try-except, dan penghapusan meminta konfirmasi. kumpulkan_wardrobe() memisahkan item menjadi atasan, bawahan, dan sepatu.

Source Code:

```
import data
from tampilan import bersih, header, jeda, tampil_item, sukses, gagal, info
from validasi import pilih_nomor, pilih_id, input_ya_tidak
from menu import pilih_menu


def kumpulkan_wardrobe():
    atasan = {}
    bawahan = {}
    sepatu = {}
```

```
    for id_item, item in data.pakaian[data.user_login].items():
        if item["jenis"] in data.baju_default:
            atasan[id_item] = item
        elif item["jenis"] in data.celana_default:
            bawahan[id_item] = item
        elif item["jenis"] in data.sepatu_default:
            sepatu[id_item] = item
    return {"atasan": atasan, "bawahan": bawahan, "sepatu": sepatu}


def tambah_item():
    bersih()
    header("TAMBAH ITEM")
    daftar_jenis = data.baju_default + data.celana_default +
data.sepatu_default
    try:
        jenis = pilih_nomor(daftar_jenis, "Pilih jenis: ")
        warna = pilih_nomor(data.warna_default, "Pilih warna: ")
    except ValueError as error:
        gagal(error)
    else:
        data.pakaian[data.user_login][data.buat_id_pakaian()] = {"jenis":
jenis, "warna": warna}
        sukses("Item berhasil ditambahkan!")
    jeda()


def lihat_wardrobe():
    bersih()
    header("WARDROBE ANDA")
    tampil_item(data.pakaian[data.user_login])
    jeda()


def ubah_item():
    bersih()
    header("UBAH ITEM")
    daftar_pakaian = data.pakaian[data.user_login]
    tampil_item(daftar_pakaian)
    if len(daftar_pakaian) > 0:
        try:
            id_item = pilih_id(daftar_pakaian, "Pilih nomor item: ")
            warna = pilih_nomor(data.warna_default, "Pilih warna baru: ")
        except ValueError as error:
            gagal(error)
        else:
            daftar_pakaian[id_item]["warna"] = warna
            sukses("Item berhasil diubah!")
```

```
    jeda()


def hapus_item():
    bersih()
    header("HAPUS ITEM")
    daftar_pakaian = data.pakaian[data.user_login]
    tampil_item(daftar_pakaian)
    if len(daftar_pakaian) > 0:
        try:
            id_item = pilih_id(daftar_pakaian, "Pilih nomor item: ")
        except ValueError as error:
            gagal(error)
        else:
            info("Yakin hapus?")
            if input_ya_tidak():
                del daftar_pakaian[id_item]
                sukses("Item berhasil dihapus!")
            else:
                info("Dibatalkan.")
    jeda()


def kelola_wardrobe():
    aktif = True
    while aktif:
        bersih()
        header("KELOLA WARDROBE")
        pilih = pilih_menu("Pilih menu", [
            ("Tambah Item", "1"),
            ("Lihat Wardrobe", "2"),
            ("Ubah Item", "3"),
            ("Hapus Item", "4"),
            ("Kembali", "0"),
        ])
        if pilih == "1":
            tambah_item()
        elif pilih == "2":
            lihat_wardrobe()
        elif pilih == "3":
            ubah_item()
        elif pilih == "4":
            hapus_item()
        else:
            aktif = False
```

I. Fitur Outfit Builder dan Lihat Outfit Berdasarkan Style

simpan_outfit() dipakai bersama untuk membuat dan mengubah outfit. Fungsi ini mengecek dulu apakah wardrobe punya minimal 1 baju, 1 celana, dan 1 sepatu, lalu semua input dibungkus satu try-except. Item yang dipilih disalin dengan .copy(), dan outfit disimpan dengan ID baru atau menimpa ID lama setelah konfirmasi. lihat_outfit_style() menampilkan set default dan outfit buatan user sesuai style yang dipilih dalam satu tabel.

Source Code:

```
from prettytable import PrettyTable
import data
from tampilan import (bersih, header, jeda, tampil_item, tampil_outfit,
tampil_detail_set,
                      tampil_pilihan, sukses, gagal, info)
from validasi import pilih_nomor, pilih_id, input_ya_tidak, input_angka
from wardrobe import kumpulkan_wardrobe
from menu import pilih_menu


def simpan_outfit(pilih_builder):
    bersih()
    header("BUAT OUTFIT" if pilih_builder == "1" else "UBAH OUTFIT")
    wardrobe = kumpulkan_wardrobe()
    atasan = wardrobe["atasan"]
    bawahan = wardrobe["bawahan"]
    sepatu = wardrobe["sepatu"]

    if len(atasan) == 0 or len(bawahan) == 0 or len(sepatu) == 0:
        gagal("Wardrobe belum lengkap!")
        gagal("Minimal punya 1 baju, 1 celana, dan 1 sepatu.")
        gagal("Tambahkan dulu di menu Kelola Wardrobe.")
        jeda()
        return

    daftar_outfit = data.outfit[data.user_login]
    id_ubah = 0

    try:
        if pilih_builder == "3":
            if len(daftar_outfit) == 0:
                raise ValueError("(Belum ada outfit)")
            tampil_outfit(daftar_outfit)
            id_ubah = pilih_id(daftar_outfit, "Pilih nomor outfit yang
diubah: ")
```

```
        nama_outfit = input("Nama outfit: ").strip()
        if nama_outfit == "":
            raise ValueError("Nama outfit tidak boleh kosong!")

        info("Pilih Atasan:")
        tampil_item(atasan)
        id_atasan = pilih_id(atasan, "Pilih atasan: ")

        info("Pilih Bawahan:")
        tampil_item(bawahan)
        id_bawahan = pilih_id(bawahan, "Pilih bawahan: ")

        info("Pilih Sepatu:")
        tampil_item(sepatu)
        id_sepatu = pilih_id(sepatu, "Pilih sepatu: ")

        info("Pilih Style Outfit:")
        style_dipilih = pilih_nomor(data.style_default, "Pilih style: ")
    except ValueError as error:
        gagal(error)
        jeda()
        return

    data_outfit = {
        "nama": nama_outfit,
        "style": style_dipilih,
        "baju": atasan[id_atasan].copy(),
        "celana": bawahan[id_bawahan].copy(),
        "sepatu": sepatu[id_sepatu].copy(),
    }

    info("Kombinasi outfit:")
    print(f"Nama  : {nama_outfit}")
    print(f"Style : {style_dipilih}")
    tampil_detail_set(data_outfit)
    info("Simpan outfit?")

    if input_ya_tidak():
        if pilih_builder == "3":
            daftar_outfit[id_ubah] = data_outfit
        else:
            daftar_outfit[data.buat_id_outfit()] = data_outfit
        sukses("Outfit berhasil disimpan!")
    else:
        info("Dibatalkan.")
    jeda()
```

```
def lihat_outfit():
    bersih()
    header("OUTFIT SAYA")
    tampil_outfit(data.outfit[data.user_login])
    jeda()


def hapus_outfit():
    bersih()
    header("HAPUS OUTFIT")
    daftar_outfit = data.outfit[data.user_login]
    tampil_outfit(daftar_outfit)
    if len(daftar_outfit) > 0:
        try:
            id_set = pilih_id(daftar_outfit, "Pilih nomor outfit: ")
        except ValueError as error:
            gagal(error)
        else:
            info("Yakin hapus?")
            if input_ya_tidak():
                del daftar_outfit[id_set]
                sukses("Outfit berhasil dihapus!")
            else:
                info("Dibatalkan.")
    jeda()


def outfit_builder():
    aktif = True
    while aktif:
        bersih()
        header("OUTFIT BUILDER")
        pilih = pilih_menu("Pilih menu", [
            ("Buat Outfit", "1"),
            ("Lihat Outfit Saya", "2"),
            ("Ubah Outfit", "3"),
            ("Hapus Outfit", "4"),
            ("Kembali", "0"),
        ])
        if pilih == "1":
            simpan_outfit("1")
        elif pilih == "2":
            lihat_outfit()
        elif pilih == "3":
            simpan_outfit("3")
        elif pilih == "4":
            hapus_outfit()
        else:
```

```
            aktif = False


def lihat_outfit_style():
    aktif = True
    while aktif:
        bersih()
        header("PILIH STYLE")
        tampil_pilihan(data.style_default)
        print("0. Kembali")
        pilih = input_angka("Pilih style: ")

        if pilih == 0:
            aktif = False
        elif pilih < 1 or pilih > len(data.style_default):
            gagal("Nomor tidak ada!")
            jeda()
        else:
            style_dipilih = data.style_default[pilih - 1]
            bersih()
            header(f"OUTFIT STYLE {style_dipilih.upper()}")
            tabel = PrettyTable(["No", "Nama", "Baju", "Celana", "Sepatu"])
            d = data.outfit_default[style_dipilih]
            tabel.add_row([
                1, "Set Default",
                f"{d['baju']['jenis']} {d['baju']['warna']}",
                f"{d['celana']['jenis']} {d['celana']['warna']}",
                f"{d['sepatu']['jenis']} {d['sepatu']['warna']}",
            ])
            nomor_urut = 1
            for s in data.outfit[data.user_login].values():
                if s["style"] == style_dipilih:
                    nomor_urut += 1
                    tabel.add_row([
                        nomor_urut, f"{s['nama']} (Buatan Anda)",
                        f"{s['baju']['jenis']} {s['baju']['warna']}",
                        f"{s['celana']['jenis']} {s['celana']['warna']}",
                        f"{s['sepatu']['jenis']} {s['sepatu']['warna']}",
                    ])
            tabel.align = "l"
            print(tabel)
            jeda()
```

J. Fungsi Validasi

input_angka() dan input_ya_tidak() bersifat rekursif, yaitu memanggil dirinya sendiri sampai input benar. validasi_username() dan validasi_password() memakai raise ValueError jika input kosong, mengandung spasi, atau (untuk username) sudah dipakai. pilih_nomor() mengembalikan isi tuple sesuai nomor yang dipilih, sedangkan pilih_id() mengubah nomor urut menjadi ID asli di dictionary.

Source Code:

```
import data
from tampilan import tampil_pilihan, gagal


def input_ya_tidak():
    jawab = input("Pilih (Ya/Tidak): ").strip().lower()
    if jawab == "ya":
        return True
    if jawab == "tidak":
        return False
    gagal("Input harus Ya atau Tidak!")
    return input_ya_tidak()


def input_angka(pesan):
    try:
        angka = int(input(pesan))
    except ValueError:
        gagal("Input harus berupa angka!")
        return input_angka(pesan)
    else:
        return angka


def validasi_username(username):
    username = username.strip()
    if username == "":
        raise ValueError("Username tidak boleh kosong!")
    if " " in username:
        raise ValueError("Username tidak boleh mengandung spasi!")
    if username.lower() in data.akun:
        raise ValueError("Username sudah dipakai!")
    return username


def validasi_password(password):
```

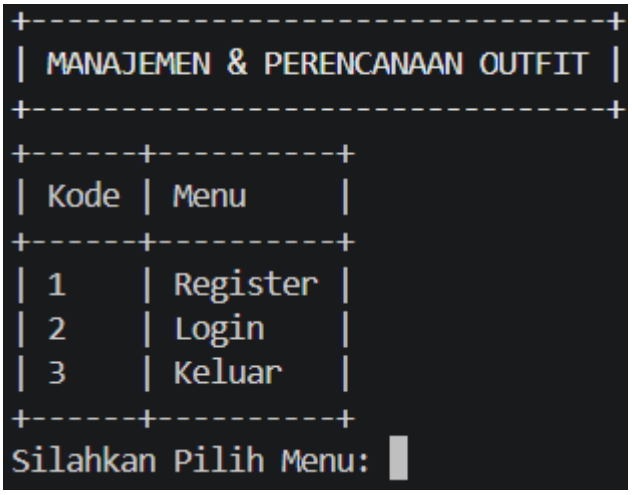
```
    password = password.strip()
    if password == "":
        raise ValueError("Password tidak boleh kosong!")
    if " " in password:
        raise ValueError("Password tidak boleh mengandung spasi!")
    return password


def pilih_nomor(daftar, pesan):
    tampil_pilihan(daftar)
    nomor = input_angka(pesan)
    if nomor < 1 or nomor > len(daftar):
        raise ValueError("Nomor tidak ada!")
    return daftar[nomor - 1]

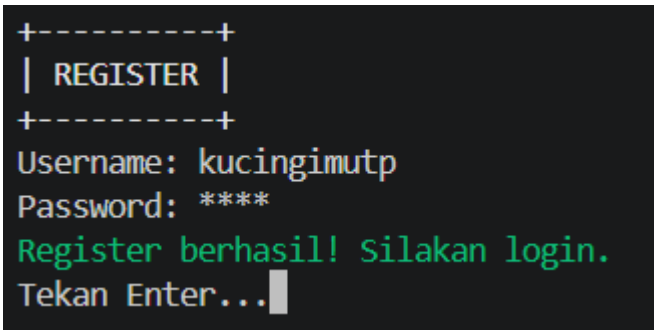

def pilih_id(daftar, pesan):
    nomor = input_angka(pesan)
    if nomor < 1 or nomor > len(daftar):
        raise ValueError("Nomor tidak ada!")
    urutan = 0
    for id_item in daftar:
        urutan += 1
        if urutan == nomor:
            return id_item
```

=

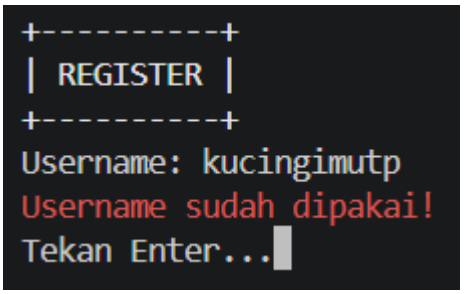
4. Hasil Output



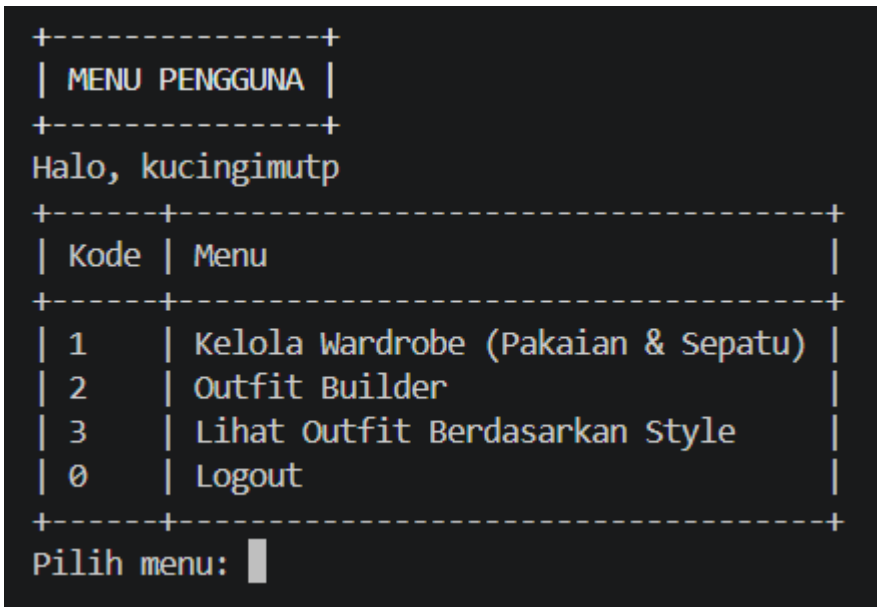
4.1 Gambar Tampilan Menu Awal.



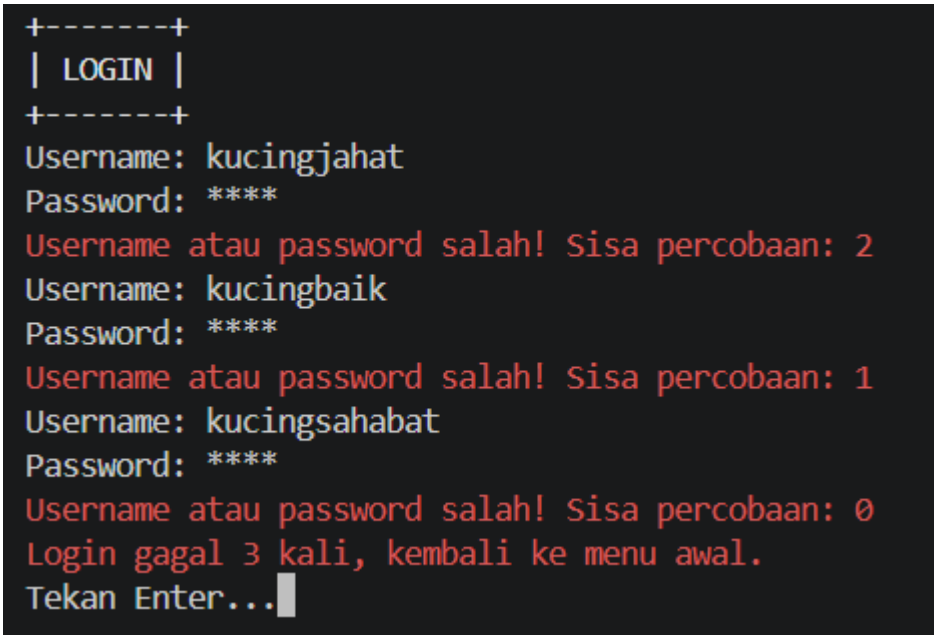
4.2 Gambar Ketika Register Berhasil.



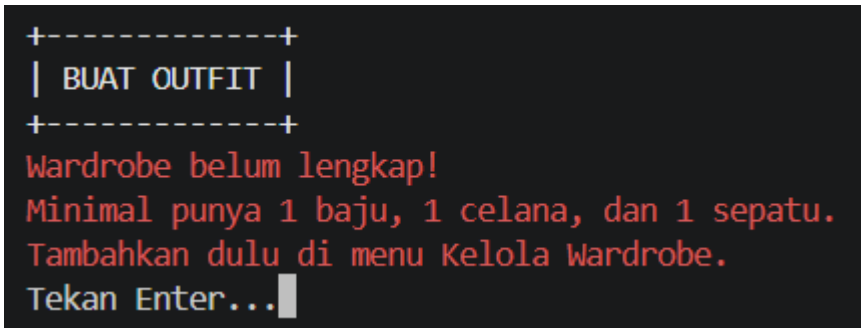
4.3 Gambar Ketika Username Sudah Dipakai.



4.4 Gambar Ketika Login Berhasil sebagai User.



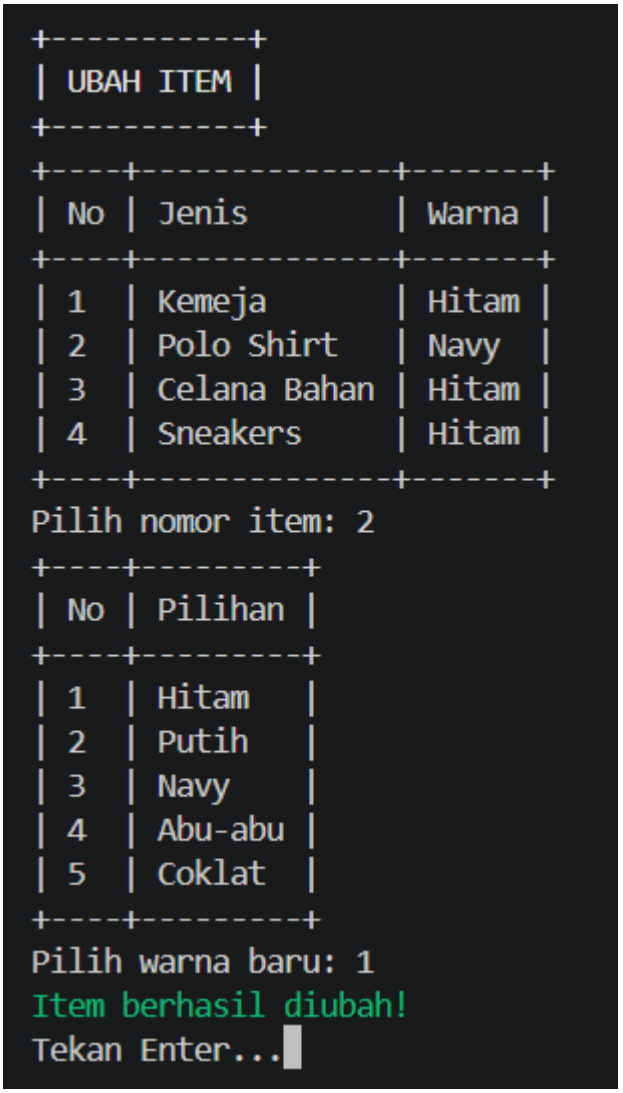
4.5 Gambar Ketika Gagal Login 3 Kali.



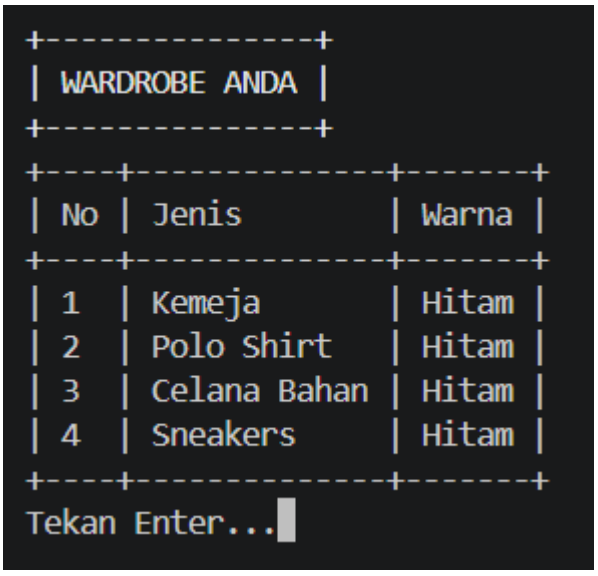
4.6 Gambar Ketika Wardrobe Belum Lengkap.



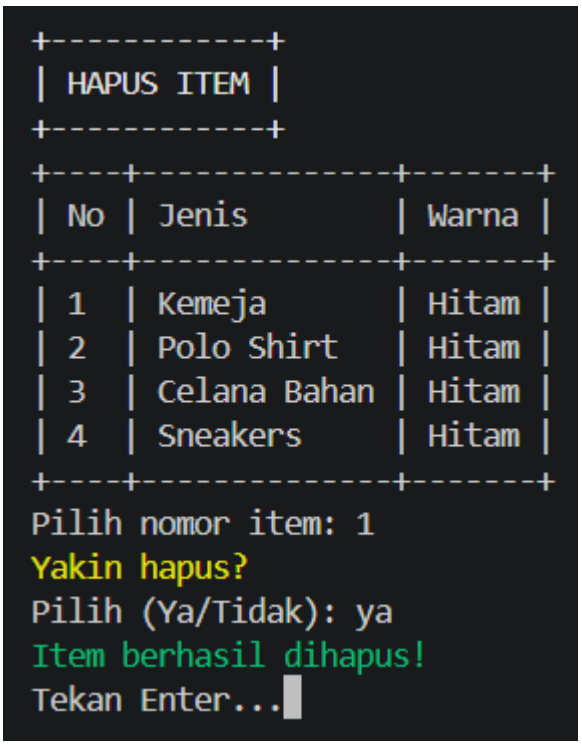
4.7 Gambar Ketika Item Berhasil Ditambahkan.



4.8 Gambar Ketika Warna Item Berhasil Diubah.



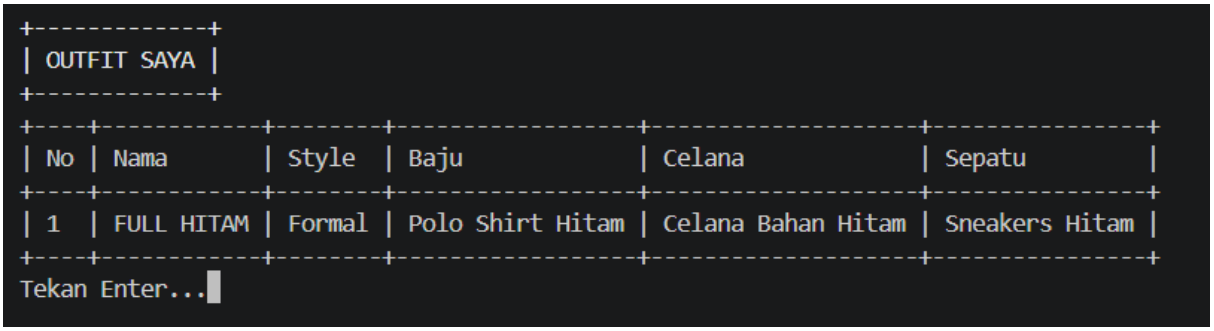
4.9 Gambar Tampilan Wardrobe Pengguna.



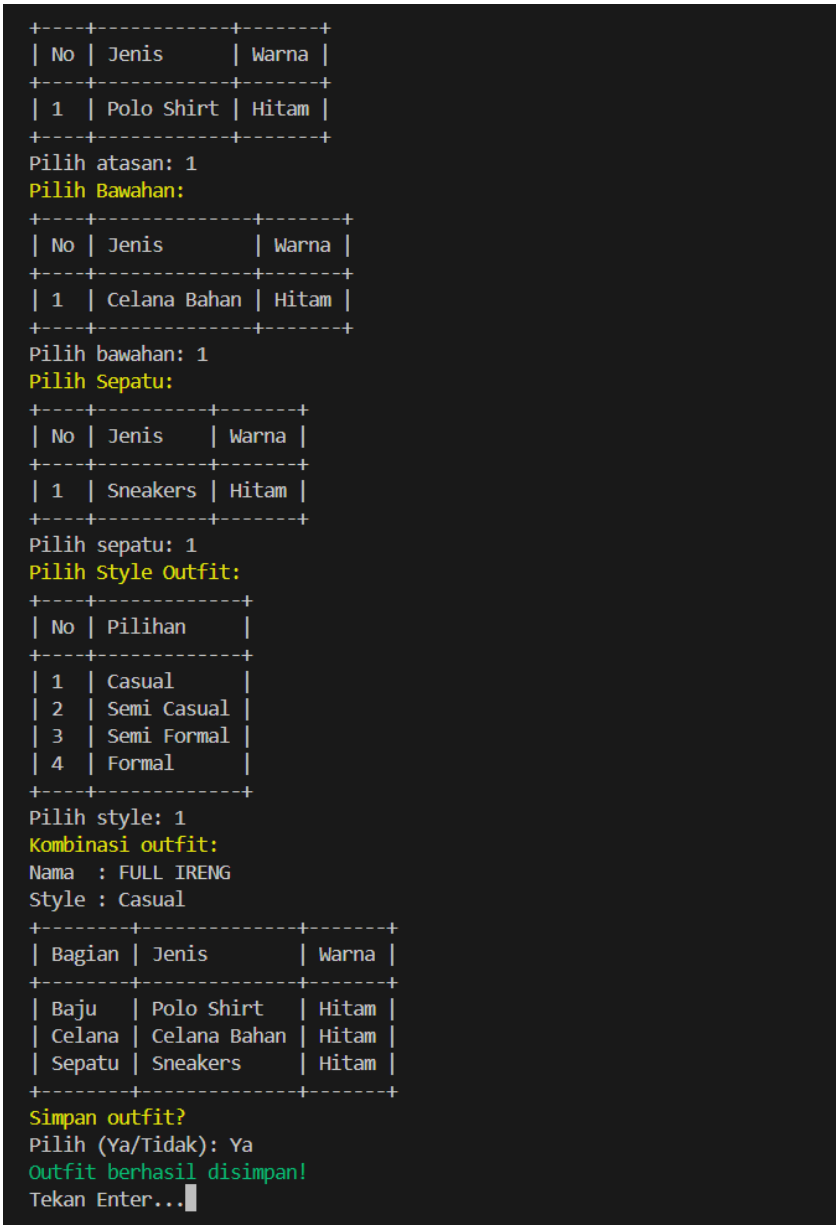
4.10 Gambar Ketika Item Berhasil Dihapus.



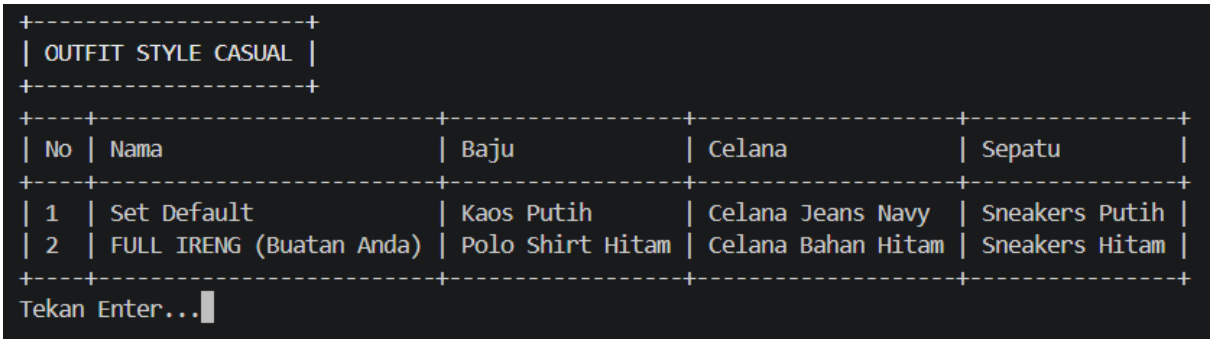
4.11 Gambar Ketika Outfit Berhasil Dibuat.



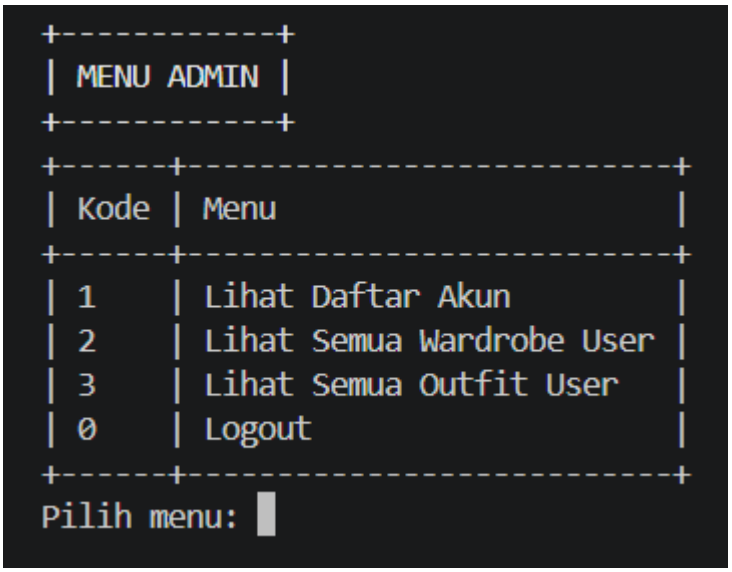
4.12 Gambar Tampilan Outfit Saya.



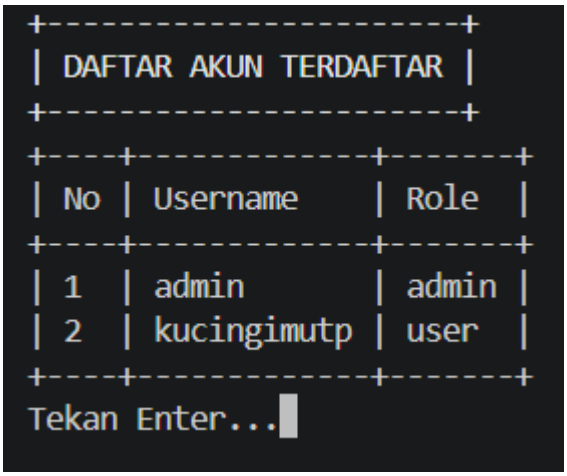
4.13 Gambar Ketika Outfit Berhasil Diubah.



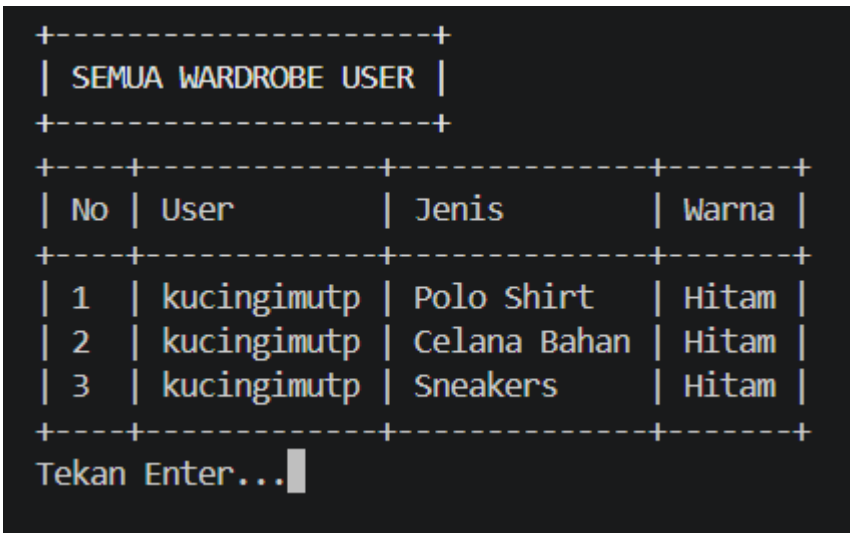
4.14 Gambar Tampilan Outfit Berdasarkan Style.



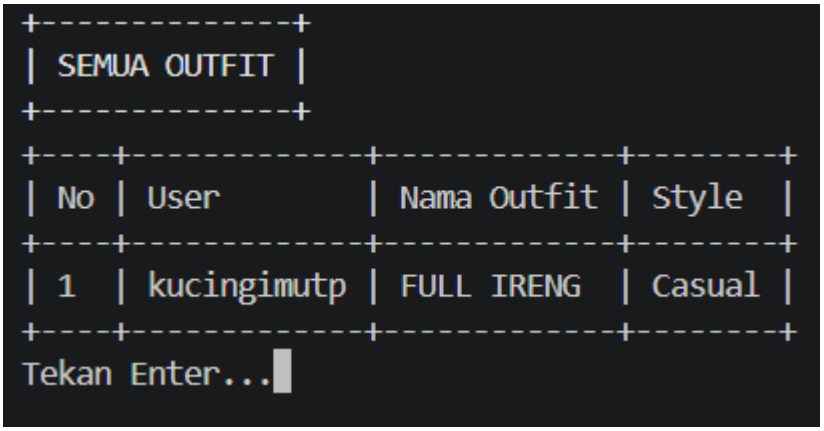
4.15 Gambar Tampilan Menu Admin.



4.16 Gambar Tampilan Daftar Akun Terdaftar.



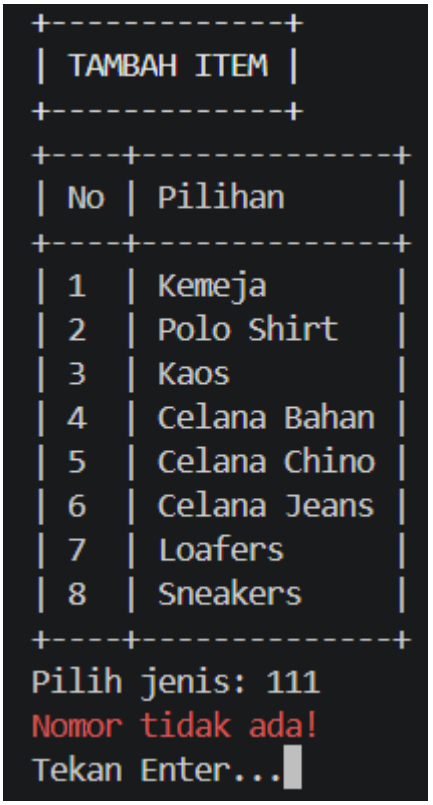
4.17 Gambar Tampilan Semua Wardrobe User.



4.18 Gambar Tampilan Semua Outfit User.



4.19 Gambar Ketika Input Bukan Angka (Rekursif input_angka).



4.20 Gambar Ketika Nomor Tidak Ada (raise ValueError).

5. Langkah-Langkah GIT

5.1 GIT ADD

```
PS C:\Users\User\Documents\praktikum-apd-2026> git add .
```

5.1 Gambar GIT ADD

Git Add berfungsi untuk menandai file yang baru dibuat atau diubah, agar file tersebut siap dimasukkan ke dalam commit berikutnya.

5.2 GIT COMMIT

```
PS C:\Users\User\Documents\praktikum-apd-2026> git commit -m "POSSTEST 8"
[main 62a009c] POSSTEST 8
 19 files changed, 668 insertions(+)
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/__pycache__/admin.cpython-313.pyc
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/__pycache__/autentikasi.cpython-313.pyc
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/__pycache__/data.cpython-313.pyc
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/__pycache__/menu.cpython-313.pyc
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/__pycache__/outfit.cpython-313.pyc
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/__pycache__/pengguna.cpython-313.pyc
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/__pycache__/tampilan.cpython-313.pyc
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/__pycache__/validasi.cpython-313.pyc
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/__pycache__/wardrobe.cpython-313.pyc
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/admin.py
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/autentikasi.py
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/data.py
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/main.py
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/menu.py
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/outfit.py
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/pengguna.py
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/tampilan.py
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/validasi.py
 create mode 100644 posttest/posstest-apd-8/2609106014-Nauvan Hadya Satrya-PT-8/wardrobe.py
```

5.2 Gambar GIT COMMIT

Git Commit berfungsi untuk menyimpan perubahan yang sudah ditandai ke riwayat repository.

5.3 GIT PUSH

```
PS C:\Users\User\Documents\praktikum-apd-2026> git push
Enumerating objects: 27, done.
Counting objects: 100% (27/27), done.
Delta compression using up to 12 threads
Compressing objects: 100% (25/25), done.
Writing objects: 100% (25/25), 21.69 KiB | 2.17 MiB/s, done.
Total 25 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/navenary/praktikum-apd-2026.git
   7d4032f..62a009c  main -> main
```

5.3 Gambar GIT PUSH

Git Push berfungsi untuk mengunggah (upload) commit yang sudah tersimpan di repository lokal ke repository di GitHub, sehingga perubahan tersebut bisa diakses secara online.